

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO NGÔN NGỮ LẬP TRÌNH JAVA
ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THIẾT BỊ ÂM THANH

Giảng viên hướng dẫn: Nguyễn Thị Hồng Anh

Thành viên nhóm đề tài

Trương Trọng Phúc - 3124410287

Chế Văn Nam - 3124410217

Nguyễn Thành Nhân - 3124410249

Hồ Minh Khuyến - 3124410168

Vũ Lê Phước Sang - 3124410303

TP Hồ Chí Minh, Tháng 3 Năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn đến cô Nguyễn Thị Hồng Anh – giảng viên bộ môn “Ngôn ngữ lập trình Java” thuộc Khoa Công Nghệ Thông Tin, trường Đại học Sài Gòn, đã trang bị cho chúng em những kiến thức, kỹ năng cơ bản cần có để có thể hoàn thành đồ án này.

Tuy nhiên, trong quá trình hoàn thiện đồ án, với vốn kiến thức cũng như kinh nghiệm còn rất khiêm tốn và là bước đầu làm quen với công việc nghiên cứu mang tính thực nghiệm, chúng em vẫn còn nhiều thiếu sót, hạn chế trong việc tìm hiểu và xây dựng đồ án về này. Rất mong được sự quan tâm, góp ý của cô để đồ án của chúng em được đầy đủ và hoàn chỉnh hơn.

Xin kính chúc cô Phan Nguyệt Minh dồi dào sức khỏe và hạnh phúc để tiếp tục thực hiện sứ mệnh cao đẹp của mình là truyền đạt kiến thức cho thế hệ mai sau.

Xin chân thành cảm ơn.

Tp.HCM ngày tháng năm 2026

Trương Trọng Phúc
Chế Văn Nam
Nguyễn Thành Nhân
Hồ Minh Khuyến
Vũ Lê Phước Sang

STT	Họ và tên	Nội dung phụ trách	Thái độ làm việc	Tỉ lệ đóng góp đề án
1	Trương Trọng Phúc	Nhập hàng, Phiếu nhập, Thống kê, Nhà cung cấp	Tích Cực	23%
2	Chế Văn Nam	Bán hàng, Phiếu xuất, Bảo hành, Phân quyền	Tích Cực	23%
3	Hồ Minh Khuyến	Sản phẩm, Phiên bản sản phẩm, Chi tiết sản phẩm	Tích Cực	23%
4	Nguyễn Thành Nhân	Nhân viên, Khách hàng	Tích Cực	17%
5	Vũ Lê Phước Sang	Đăng nhập, Khuyến Mãi	Tích Cực	14%

Mục lục

LỜI CẢM ƠN	2
Chương 1: Mô tả về bài toán, công cụ sử dụng.....	10
1.1 Bối cảnh	10
1.2 Ý tưởng thực hiện	10
1.3 Các công cụ được sử dụng	11
1.4 Bố trí dự án.....	12
pom.xml	12
src/main/java	12
src/test/java.....	13
Các file khác	13
Chương 2: Phân tích thiết kế bài toán	13
2.1 Tổng quan.....	13
2.2 Mô hình quan hệ ERD.....	14
2.3 Mô tả tổng quan các quan hệ.....	14
2.3.1 Nhóm phân quyền – tài khoản.....	14
2.3.2 Nhóm người dùng	15
2.3.3 Nhóm danh mục sản phẩm	15
2.3.4 Nhóm nghiệp vụ nhập – xuất kho	15
2.3.5 Nhóm đối trả – bảo hành.....	15
2.4 Tổng thể :.....	15
Chương 3: Chương trình đồ án	16
3.1 Cơ sở dữ liệu.....	16
3.1.1 Tổng quan chung	16

3.1.2 Bảng bảo hành.....	16
3.2.3 Bảng chi tiết bảo hành.....	17
3.2.4 Bảng chi tiết phiếu nhập.....	17
3.2.5 Bảng chi tiết phiếu xuất.....	18
3.2.6 Bảng chi tiết quyền	18
3.2.7 Bảng chi tiết sản phẩm	19
3.2.8 Bảng chức năng.....	19
3.2.9 Bảng con người	19
3.2.10 Bảng hãng sản xuất.....	20
3.2.11 Bảng khách hàng.....	20
3.2.12 Bảng khuyến mãi	20
3.2.13 Bảng loại sản phẩm.....	21
3.2.14 Bảng NCC_SanPham	21
3.2.15 Bảng nhà cung cấp.....	21
3.2.16 Bảng nhân viên.....	22
3.2.17 Bảng nhóm quyền	22
3.2.18 Bảng phiên bản sản phẩm.....	23
3.2.19 Bảng phiếu nhập	23
3.2.20 Bảng phiếu xuất	24
3.2.21 Bảng sản phẩm.....	24
3.2.22 Bảng tài khoản	25
3.2 Giao diện.....	26
3.2.1 Đăng nhập.....	26
3.2.2 Trang chủ.....	27
3.2.3 Sản phẩm	28

3.2.4 Bán hàng	31
3.2.5 Nhập hàng:	35
3.2.6 Phiếu nhập	40
3.2.7 Phiếu xuất	41
3.2.8 Khuyến mãi	42
3.2.9 Bảo hành	43
3.2.10 Nhà cung cấp	44
3.2.11 Nhân viên	45
3.2.12 Khách hàng	46
3.2.13 Thống kê	47
3.2.14 Phân quyền	48
3.2.15 Đổi trả	49
Phụ lục: Code demo chương trình	52
1. ChiTietInterface:	52
2. DAOInterface:	52
3. BaoHanhDAO	52
4. ChiTietBaoHanhDAO	56
5. ChiTietPhieuNhapDAO	58
6. ChiTietPhieuXuatDAO	60
7. ChiTietQuyenDAO	61
8. ChiTietSanPhamDAO	63
9. ChucNangDAO	67
10. ConNguoiDAO	68
11. KhachHangDAO	69
12. DoiTraDAO	71

13.	KhuyenMaiDAO	79
14.	LoaiSPDAO	81
15.	NhaCungCapDAO	81
16.	NhanVienDAO	84
17.	NhomQuyềnDAO	88
18.	PhieuNhapDAO	91
19.	PhieuXuatDAO	94
20.	SanPhamDAO	101
21.	PhienBanSanPhamDAO	104
22.	TaiKhoanDAO	113
23.	ThongKeDAO	115
	Tài liệu tham khảo	119

Mục lục hình ảnh

Hình 1	Mô hình quan hệ ERD	14
Hình 2	Bảng tổng quan chung	16
Hình 3	Bảng bảo hành	16
Hình 4	Bảng chi tiết bảo hành	17
Hình 5	Bảng chi tiết phiếu nhập	17
Hình 6	Bảng chi tiết phiếu xuất	18
Hình 7	Bảng chi tiết quyền	18

Hình 8 Bảng chi tiết sản phẩm.....	19
Hình 9 Bảng chức năng	19
Hình 10 Bảng con người.....	19
Hình 11 Bảng hãng sản xuất.....	20
Hình 12 Bảng khách hàng.....	20
Hình 13 Bảng khuyến mãi	20
Hình 14 Bảng loại sản phẩm.....	21
Hình 15 Bảng NCC_Sản phẩm.....	21
Hình 16 Bảng nhà cung cấp.....	21
Hình 17 Bảng nhân viên	22
Hình 18 Bảng nhóm quyền.....	22
Hình 19 Bảng phiên bản sản phẩm	23
Hình 20 Bảng phiếu nhập	23
Hình 21 Bảng phiếu xuất	24
Hình 22 Bảng sản phẩm.....	24
Hình 23 Bảng tài khoản	25
Hình 24 Bảng đổi trả.....	25
Hình 25 Giao diện đăng nhập hệ thống	26
Hình 26 Giao diện trang chủ hệ thống.....	27
Hình 27 Giao diện quản lý danh mục sản phẩm	28
Hình 28 Hộp thoại thêm thông tin sản phẩm	29
Hình 29 Xuất dữ liệu sản phẩm ra file Excel.....	30
Hình 30 Giao diện bán hàng(Tạo giỏ hàng)	31
Hình 31 Chọn sản phẩm cần bán	32
Hình 32 Hộp thoại chọn khách hàng hoặc thêm khách hàng mới	32
Hình 33 Hộp thoại xác nhận bán hàng.....	33
Hình 34 Hộp thoại chọn khuyến mãi còn hạn	34
Hình 35 Hóa đơn xuất phiếu bán hàng(File PDF)	35
Hình 36 Chọn sản phẩm để nhập hàng vào kho	35
Hình 37 Hộp thoại xác nhận nhập hàng vào kho.....	36
Hình 38 Hộp thoại xác nhận nhập hàng vào kho.....	37

Hình 39 Nhập hàng hàng loạt từ file Excel	39
Hình 40 Xuất hóa đơn nhập ra file PDF	39
Hình 41 Giao diện quản lý danh sách phiếu nhập	40
Hình 42 Giao diện quản lý danh sách phiếu xuất	41
Hình 43 Giao diện quản lý các chương trình khuyến mãi	42
Hình 44 Giao diện quản lý thông tin bảo hành thiết bị.....	43
Hình 45 Giao diện quản lý thông tin nhà cung cấp	44
Hình 46 Giao diện quản lý thông tin nhân viên.....	45
Hình 47 Giao diện thông tin khách hàng	46
Hình 48 Giao diện bảng điều khiển, thống kê tổng quan	47
Hình 49 Giao diện cài đặt và phân quyền người dùng	48
Hình 50 Giao diện quản lý yêu cầu đổi trả sản phẩm.....	49
Hình 51 Hộp thoại thêm đổi trả cho sản phẩm lỗi.....	50

Chương 1: Mô tả về bài toán, công cụ sử dụng

1.1 Bối cảnh

Trong bối cảnh nhu cầu giải trí tại gia và xu hướng tiêu dùng các thiết bị âm thanh ngày càng tăng, các cửa hàng kinh doanh sản phẩm âm thanh (loa, tai nghe, ampli, micro, phụ kiện...) phải xử lý khối lượng giao dịch lớn và đa dạng. Mỗi ngày, cửa hàng tiếp nhận nhiều nhà cung cấp, nhiều dòng sản phẩm với thông số kỹ thuật khác nhau, các phiên bản theo màu sắc/đời máy, cùng các chương trình khuyến mãi theo thời điểm. Song song đó là các nghiệp vụ bán hàng, xuất hóa đơn, quản lý tồn kho, đổi trả, và bảo hành. Nếu vận hành thủ công bằng sổ sách hoặc file rời rạc, cửa hàng dễ gặp tình trạng sai lệch số liệu, thất thoát hàng hóa, chậm cập nhật tồn kho, khó truy vết lịch sử bán–nhập, và mất nhiều thời gian tổng hợp báo cáo doanh thu.

Từ thực tế đó, bài toán đặt ra là xây dựng một hệ thống quản lý cửa hàng âm thanh nhằm hỗ trợ số hóa quy trình vận hành. Hệ thống cần cho phép quản lý danh mục sản phẩm theo nhóm/loại, hãng sản xuất, thông tin giá bán, giá nhập, và trạng thái kinh doanh; quản lý khách hàng, nhân viên và tài khoản đăng nhập theo quyền hạn; quản lý nhà cung cấp và các phiếu nhập–xuất; ghi nhận chi tiết từng giao dịch để đảm bảo tính minh bạch và có thể đối soát. Ngoài ra, vì sản phẩm âm thanh thường đi kèm chính sách bảo hành và phát sinh đổi trả khi lỗi kỹ thuật, hệ thống cũng cần lưu vết thông tin bảo hành, thời gian áp dụng, và lịch sử xử lý để giảm tranh chấp và nâng cao chất lượng dịch vụ.

Lý do thực hiện đề tài xuất phát từ nhu cầu tối ưu hóa hiệu suất làm việc và nâng cao độ chính xác dữ liệu. Khi có phần mềm quản lý tập trung, nhân viên có thể thao tác nhanh hơn trong khâu bán hàng (tìm sản phẩm, áp khuyến mãi, chọn khách hàng), đồng thời tồn kho được cập nhật tự động sau mỗi lần nhập/xuất. Người quản lý có thể theo dõi tình hình kinh doanh theo ngày/tháng, nắm được sản phẩm bán chạy, doanh thu, lợi nhuận ước tính và hiệu quả chương trình khuyến mãi, từ đó ra quyết định nhập hàng hoặc điều chỉnh giá kịp thời. Về lâu dài, hệ thống còn giúp chuẩn hóa quy trình, giảm rủi ro sai sót do nhập liệu thủ công, và tạo nền tảng để mở rộng thêm các chức năng như xuất báo cáo, phân tích dữ liệu, hoặc tích hợp thanh toán.

Tóm lại, bài toán “quản lý cửa hàng âm thanh” không chỉ đơn thuần là lưu trữ thông tin sản phẩm, mà là một hệ thống hỗ trợ toàn bộ chuỗi nghiệp vụ từ nhập hàng, bán hàng, khách hàng, nhân sự, khuyến mãi đến bảo hành/đổi trả và thống kê. Việc xây dựng hệ thống này là cần thiết để giúp cửa hàng hoạt động hiệu quả, minh bạch, và đáp ứng tốt hơn nhu cầu của khách hàng trong môi trường kinh doanh cạnh tranh.

1.2 Ý tưởng thực hiện

Sau khi xác định bối cảnh và các khó khăn khi quản lý thủ công, nhóm bắt đầu đặt câu hỏi: “Nếu là người bán hàng/ quản lý cửa hàng âm thanh, mình cần thao tác gì nhanh

nhất mỗi ngày?”. Từ đó, ý tưởng cốt lõi của project được hình thành: xây dựng một website vừa hỗ trợ bán hàng trực tuyến cho khách, vừa là công cụ quản trị nội bộ giúp cửa hàng kiểm soát hàng hóa và doanh thu theo thời gian thực.

Quá trình suy nghĩ được triển khai theo hướng “đi từ nghiệp vụ đến tính năng”. Nhóm phác thảo luồng hoạt động cơ bản: khách xem sản phẩm → tìm kiếm/lọc theo loại, hãng, mức giá → xem chi tiết (thông số, hình ảnh, bảo hành) → thêm vào giỏ → đặt hàng. Với phía quản trị, nhóm ưu tiên các thao tác ảnh hưởng trực tiếp đến vận hành: quản lý sản phẩm và phiên bản (màu/đời), cập nhật tồn kho, tạo–duyet đơn hàng, quản lý khách hàng, phân quyền nhân viên, và theo dõi khuyến mãi. Vì mặt hàng âm thanh có nhiều thông số kỹ thuật và dễ phát sinh đổi trả/bảo hành, nhóm quyết định bổ sung module bảo hành/đổi trả để lưu lịch sử xử lý, giúp tra cứu nhanh khi khách liên hệ.

Tiếp theo, nhóm chuyển từ ý tưởng sang thiết kế dữ liệu: xác định các thực thể chính (Sản phẩm, Loại sản phẩm, Hãng, Khách hàng, Nhân viên, Đơn hàng, Chi tiết đơn, Khuyến mãi...) và mối quan hệ giữa chúng để đảm bảo truy vấn thông kê thuận lợi. Sau đó mới đến thiết kế giao diện theo nguyên tắc đơn giản, ít bước: thanh tìm kiếm nổi bật, bộ lọc rõ ràng, trang quản trị dạng bảng có phân trang và thao tác nhanh. Cuối cùng, nhóm đặt mục tiêu “dễ mở rộng”: tách riêng phần hiển thị và xử lý nghiệp vụ, để sau này có thể thêm tính năng như đánh giá sản phẩm, thanh toán online hoặc báo cáo nâng cao mà không phải làm lại toàn bộ hệ thống.

1.3 Các công cụ được sử dụng

Thành phần	Chi tiết
Ngôn Ngữ	Java JDK 21+
Build Tool	Apache Maven
Giao Diện	Java Swing
Database	MySQL (XAMPP)
Kết Nối JDBC	(Java Database Connectivity)

1.4 Bố trí dự án

Dự án được tổ chức theo chuẩn Maven, tách rõ mã nguồn chính, tài nguyên và mã test, đồng thời chia backend/frontend trong src/main/java:

pom.xml

- **Vai trò:** File cấu hình Maven, khai báo dependency, phiên bản JDK, plugin build, tên project...
- **Ý nghĩa:** Giúp tự động hóa quá trình build, test, đóng gói và quản lý thư viện.

src/main/java

Backend

- **DTO:** Chứa các lớp mô tả dữ liệu (Data Transfer Object) như SanPham, KhachHang, NhanVien, PhieuNhap, PhieuXuat, các lớp thống kê, thuộc tính sản phẩm... Đây là mô hình hóa bảng trong CSDL thành đối tượng Java.
- **DAO:** Chứa lớp truy xuất dữ liệu (Data Access Object) như SanPhamDAO, KhachHangDAO, PhieuNhapDAO, ChiTietPhieuXuatDAO, ThongKeDAO... chịu trách nhiệm làm việc trực tiếp với CSDL (SQL, kết nối, CRUD).
- **BUS:** Chứa lớp xử lý nghiệp vụ (Business) như SanPhamBUS, KhachHangBUS, NhanVienBUS, PhieuNhapBUS, ThongKeBUS... nơi cài đặt logic nghiệp vụ, kiểm tra ràng buộc, phối hợp DAO và DTO.
- **DatabaseHelper:** Lớp trợ giúp kết nối/đóng kết nối CSDL, đọc cấu hình.
- **Ý nghĩa:** Tách riêng **tầng dữ liệu (DAO)**, **tầng nghiệp vụ (BUS)** và **tầng mô hình (DTO)** giúp code dễ bảo trì, thay đổi CSDL hoặc logic mà ít ảnh hưởng lẫn nhau.

Frontend

- **Component:** Chứa các thành phần giao diện dùng chung như nút bấm (CustomButton, ButtonAdd, ButtonFix...), bảng (Table), component xuất Excel/PDF, dialog thao tác... giúp tái sử dụng giao diện.
- **GUI:** Chia theo module nghiệp vụ:
 - BanHang, SanPham, KhachHang, NhanVien, NhaCungCap, KhuyenMai, BaoHang, PhieuNhap, PhieuXuat, ThongKe, Login, Navigation...
 - Mỗi thư mục chứa Panel, Table, Toolbar, Dialog tương ứng cho màn hình chính, bảng dữ liệu, thanh công cụ, hộp thoại thêm/sửa/xem chi tiết.
- **Ý nghĩa:** Tách giao diện theo chức năng giúp dễ phát triển, sửa một module (ví dụ KhachHang) không ảnh hưởng module khác.

src/main/resources

- **images/icon:** Chứa icon giao diện (search, refresh, shopping-basket, user, ticket...).
- **db.properties:** File cấu hình kết nối CSDL (url, user, password...).
- **Ý nghĩa:** Lưu trữ tài nguyên không phải code nhưng cần khi chạy (hình ảnh, cấu hình), Maven tự đóng gói kèm theo.

src/test/java

- Chứa các lớp test như AppTest.java, dùng để viết unit test/kiểm tra chức năng.

Các file khác

- **Script.sql:** Script tạo CSDL/bảng, dữ liệu mẫu phục vụ cài đặt ban đầu.
- **Gitignore:** Khai báo các file/thư mục không đưa lên Git (như target, file build, file tạm).

Cách bố trí này đảm bảo dự án **rõ ràng theo tầng (DTO–DAO–BUS–GUI), phân tách backend/frontend**, dễ mở rộng chức năng mới và thuận tiện cho việc bảo trì, làm việc nhóm.

Chương 2: Phân tích thiết kế bài toán

2.1 Tổng quan

Bảng	Mô tả
Sản phẩm	Quản lý thông tin sản phẩm, phiên bản sản phẩm, chi tiết sản phẩm
Con người	Quản lý thông tin nhân viên và khách hàng
Hóa đơn (nhập/xuất)	Lịch sử giao dịch và chi tiết hóa đơn
Phân quyền	Quản lý quyền có thể tác động khi đăng nhập

2.3.2 Nhóm người dùng

- **NhanVien** và **KhachHang** là các thực thể người dùng nghiệp vụ; dữ liệu đăng nhập/thuộc quyền được quản lý qua **TaiKhoan**.
- **KhachHang** phát sinh giao dịch mua và được tham chiếu trong **PhieuXuat**.

2.3.3 Nhóm danh mục sản phẩm

- **SanPham** là trung tâm, thuộc **LoaiSP** và **HangSX** (một loại/hãng có nhiều sản phẩm).
- **NhaCungCap** liên kết **SanPham** qua bảng **NCC_SanPham** để biểu diễn quan hệ n-n (một NCC cung cấp nhiều SP và một SP có thể lấy từ nhiều NCC).

2.3.4 Nhóm nghiệp vụ nhập – xuất kho

- **PhieuNhap** (phiếu nhập) gắn với **NhaCungCap** và **NhanVien**; chi tiết từng mặt hàng trong phiếu nằm ở **CTPhieuNhap** (1-n: 1 phiếu có nhiều dòng chi tiết).
- **PhieuXuat** (phiếu xuất/bán) gắn với **KhachHang**, **NhanVien** và có thể áp **KhuyenMai**; chi tiết bán nằm ở **CTPhieuXuat** (1-n).

2.3.5 Nhóm đổi trả – bảo hành

- **DoiTra** ghi nhận yêu cầu đổi/trả phát sinh từ giao dịch bán (liên hệ tới **PhieuXuat/sản phẩm**).
- **BaoHanh** lưu thông tin bảo hành theo sản phẩm/phiếu; chi tiết từng mục bảo hành nằm ở **CTBaoHanh** (1-n).

2.4 Tổng thể :

ERD thể hiện rõ luồng dữ liệu: Danh mục (Sản phẩm–Loại–Hãng–NCC) → Nhập kho (Phiếu nhập–CT) → Bán hàng (Phiếu xuất–CT–Khuyến mãi–Khách hàng) → Hậu mãi (Đổi trả/Bảo hành), đồng thời có lớp tài khoản–phân quyền để kiểm soát truy cập hệ thống.

Chương 3: Chương trình đồ án

3.1 Cơ sở dữ liệu

3.1.1 Tổng quan chung

Table	Action	Rows	Type	Collation	Size	Overhead
☐ baohanh	★ Browse Structure Search Insert Empty Drop	224	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ chitietbaohanh	★ Browse Structure Search Insert Empty Drop	224	InnoDB	utf8mb4_general_ci	80.0 KiB	-
☐ chitietphieunhap	★ Browse Structure Search Insert Empty Drop	85	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ chitietphieuxuat	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ chitietquyen	★ Browse Structure Search Insert Empty Drop	68	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ chitietsp	★ Browse Structure Search Insert Empty Drop	8,353	InnoDB	utf8mb4_general_ci	2.0 MiB	-
☐ chucnang	★ Browse Structure Search Insert Empty Drop	12	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ connguoi	★ Browse Structure Search Insert Empty Drop	9	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ hangsx	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ khachhang	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ khuyenmai	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ loaisp	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ ncc_sanpham	★ Browse Structure Search Insert Empty Drop	26	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ nhacungcap	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ nhanvien	★ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ nhomquyen	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ phienbansp	★ Browse Structure Search Insert Empty Drop	28	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ phieunhap	★ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	64.0 KiB	-
☐ phieuxuat	★ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	80.0 KiB	-
☐ sanpham	★ Browse Structure Search Insert Empty Drop	28	InnoDB	utf8mb4_general_ci	64.0 KiB	-
☐ taikhoan	★ Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	48.0 KiB	-

Hình 2 Bảng tổng quan chung

3.1.2 Bảng bảo hành

#	Name	Type	Collation	Attributes	Null	Default
☐ 1	MaBH 🔑	varchar(20)	utf8mb4_general_ci		No	None
☐ 2	Malmei 🔑	varchar(50)	utf8mb4_general_ci		Yes	NULL
☐ 3	MaPhieuXuat 🔑	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐ 4	NgayBatDau	date			Yes	curdate()
☐ 5	NgayKetThuc	date			Yes	NULL
☐ 6	TinhTrang	varchar(50)	utf8mb4_general_ci		Yes	Còn bảo hành
☐ 7	TrangThai	tinyint(1)			Yes	1

Hình 3 Bảng bảo hành

3.2.3 Bảng chi tiết bảo hành

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaCTBH 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaBH 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	3	NoiDung	text	utf8mb4_general_ci		Yes	NULL
☐	4	TinhTrang	varchar(50)	utf8mb4_general_ci		Yes	Đang sửa chữa

Hình 4 Bảng chi tiết bảo hành

3.2.4 Bảng chi tiết phiếu nhập

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaPhieuNhap 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaPhienBan  	varchar(20)	utf8mb4_general_ci		No	None
☐	3	SoLuong	int(11)			Yes	NULL
☐	4	DonGia	double			Yes	NULL
☐	5	ThanhTien	double			Yes	NULL

Hình 5 Bảng chi tiết phiếu nhập

3.2.5 Bảng chi tiết phiếu xuất

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaPhieuXuat 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaPhienBan 	varchar(20)	utf8mb4_general_ci		No	None
☐	3	SoLuong	int(11)			Yes	NULL
☐	4	DonGia	double			Yes	NULL
☐	5	ThanhTien	double			Yes	NULL

Hình 6 Bảng chi tiết phiếu xuất

3.2.6 Bảng chi tiết quyền

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaNhomQuyền 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaChucNang 	varchar(50)	utf8mb4_general_ci		No	None
☐	3	HanhDong 	varchar(20)	utf8mb4_general_ci		No	None

Hình 7 Bảng chi tiết quyền

3.2.7 Bảng chi tiết sản phẩm

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	Malmei 	varchar(50)	utf8mb4_general_ci		No	None
☐	2	MaPhienBan 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	3	MaPhieuNhap	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	4	MaPhieuXuat	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	5	TinhTrang 	varchar(50)	utf8mb4_general_ci		Yes	Trong kho
☐	6	TrangThai	tinyint(1)			Yes	1

Hình 8 Bảng chi tiết sản phẩm

3.2.8 Bảng chức năng

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaChucNang 	varchar(50)	utf8mb4_general_ci		No	None
☐	2	TenChucNang	varchar(100)	utf8mb4_general_ci		No	None
☐	3	MoTa	text	utf8mb4_general_ci		Yes	NULL

Hình 9 Bảng chức năng

3.2.9 Bảng con người

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	ID 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	HoTen	varchar(40)	utf8mb4_general_ci		No	None
☐	3	SĐT 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	4	DiaChi	varchar(100)	utf8mb4_general_ci		Yes	NULL

Hình 10 Bảng con người

3.2.10 Bảng hãng sản xuất

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaHang 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	TenHang	varchar(50)	utf8mb4_general_ci		No	None
☐	3	QuocGia	varchar(50)	utf8mb4_general_ci		Yes	NULL

Hình 11 Bảng hãng sản xuất

3.2.11 Bảng khách hàng

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	ID 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	TrangThai	tinyint(1)			Yes	1

Hình 12 Bảng khách hàng

3.2.12 Bảng khuyến mãi

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaKM 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	TenKM	varchar(100)	utf8mb4_general_ci		Yes	NULL
☐	3	DieuKienGiam	double			Yes	NULL
☐	4	PhanTramGiam	double			Yes	NULL
☐	5	NgayBatDau	date			Yes	NULL
☐	6	NgayKetThuc	date			Yes	NULL
☐	7	TrangThai	int(11)			Yes	1

Hình 13 Bảng khuyến mãi

3.2.13 Bảng loại sản phẩm

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaLoai 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	TenLoai	varchar(50)	utf8mb4_general_ci		No	None

Hình 14 Bảng loại sản phẩm

3.2.14 Bảng NCC_SanPham

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaNCC 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaSP  	varchar(20)	utf8mb4_general_ci		No	None

Hình 15 Bảng NCC_Sản phẩm

3.2.15 Bảng nhà cung cấp

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaNCC 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaSP  	varchar(20)	utf8mb4_general_ci		No	None

Hình 16 Bảng nhà cung cấp

3.2.16 Bảng nhân viên

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	ID 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	ChucVu	varchar(50)	utf8mb4_general_ci		Yes	NULL
☐	3	Email	varchar(100)	utf8mb4_general_ci		Yes	NULL
☐	4	Luong	double			Yes	NULL
☐	5	TrangThai	tinyint(1)			Yes	1

Hình 17 Bảng nhân viên

3.2.17 Bảng nhóm quyền

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaNhomQuyen 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	TenNhomQuyen	varchar(100)	utf8mb4_general_ci		Yes	NULL
☐	3	MoTa	text	utf8mb4_general_ci		Yes	NULL

Hình 18 Bảng nhóm quyền

3.2.18 Bảng phiên bản sản phẩm

#	Name	Type	Collation	Attributes	Null	Default
☐ 1	MaPhienBan 	varchar(20)	utf8mb4_general_ci		No	None
☐ 2	MaSP 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐ 3	MauSac	varchar(50)	utf8mb4_general_ci		Yes	NULL
☐ 4	CongSuat	varchar(50)	utf8mb4_general_ci		Yes	NULL
☐ 5	Pin	varchar(50)	utf8mb4_general_ci		Yes	NULL
☐ 6	KetNoi	varchar(100)	utf8mb4_general_ci		Yes	NULL
☐ 7	GiaNhap	double			Yes	NULL
☐ 8	GiaBan	double			Yes	NULL
☐ 9	SoLuongTon	int(11)			Yes	0
☐ 10	TrangThai	tinyint(1)			Yes	1
☐ 11	HinhAnh	varchar(255)	utf8mb4_general_ci		Yes	NULL

Hình 19 Bảng phiên bản sản phẩm

3.2.19 Bảng phiếu nhập

#	Name	Type	Collation	Attributes	Null	Default
☐ 1	MaPhieuNhap 	varchar(20)	utf8mb4_general_ci		No	None
☐ 2	NgayNhap 	datetime			Yes	current_timestamp()
☐ 3	MaNV 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐ 4	MaNCC 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐ 5	TongTien	double			Yes	0
☐ 6	TrangThai	tinyint(1)			Yes	1

Hình 20 Bảng phiếu nhập

3.2.20 Bảng phiếu xuất

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaPhieuXuat 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	NgayXuat 	datetime			Yes	current_timestamp()
☐	3	MaNV 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	4	MaKH 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	5	MaKM 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	6	TongTien	double			Yes	0
☐	7	TrangThai	tinyint(1)			Yes	1

Hình 21 Bảng phiếu xuất

3.2.21 Bảng sản phẩm

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaSP 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	TenSP 	varchar(100)	utf8mb4_general_ci		No	None
☐	3	MaLoai 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	4	MaHang 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	5	MoTa	text	utf8mb4_general_ci		Yes	NULL
☐	6	ThoiGianBaoHanh	int(11)			Yes	12
☐	7	TrangThai	tinyint(1)			Yes	1
☐	8	HinhAnh	varchar(255)	utf8mb4_general_ci		Yes	NULL

Hình 22 Bảng sản phẩm

3.2.22 Bảng tài khoản

		Table structure	Relation view				
	#	Name	Type	Collation	Attributes	Null	Default
☐	1	TenDangNhap 	varchar(50)	utf8mb4_general_ci		No	None
☐	2	MatKhau	varchar(255)	utf8mb4_general_ci		No	None
☐	3	MaNV 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	4	MaNhomQuyzen 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	5	TrangThai	tinyint(1)			Yes	1

Hình 23 Bảng tài khoản

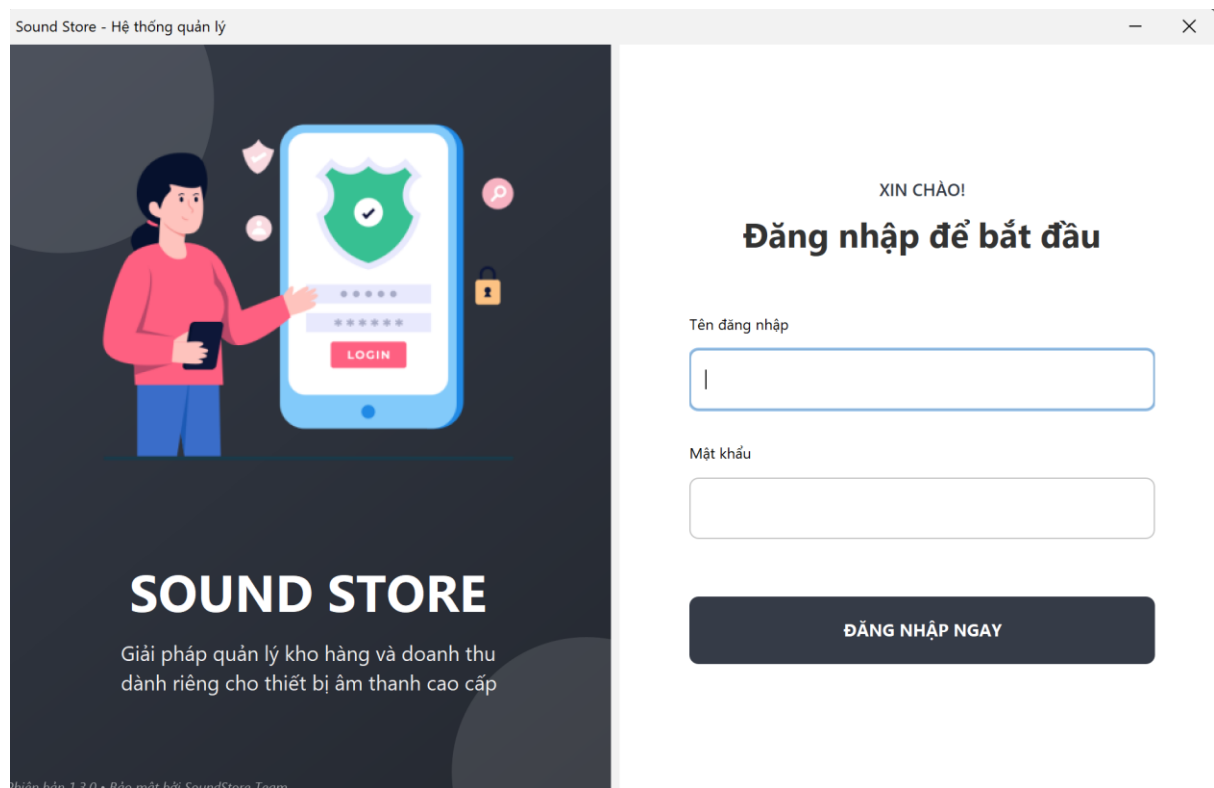
3.2.23 Bảng đổi trả

	#	Name	Type	Collation	Attributes	Null	Default
☐	1	MaDoiTra 	varchar(20)	utf8mb4_general_ci		No	None
☐	2	MaKH 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	3	MaPhieuXuat 	varchar(20)	utf8mb4_general_ci		Yes	NULL
☐	4	Malmei 	varchar(50)	utf8mb4_general_ci		Yes	NULL
☐	5	NgayDoiTra	date			Yes	NULL
☐	6	LyDo	text	utf8mb4_general_ci		Yes	NULL
☐	7	TrangThai	tinyint(1)			Yes	1

Hình 24 Bảng đổi trả

3.2 Giao diện

3.2.1 Đăng nhập



Hình 25 Giao diện đăng nhập hệ thống

Khi mở ứng dụng, người dùng vào màn hình LoginFrame và nhập tài khoản/mật khẩu. Hệ thống gửi thông tin này xuống tầng BUS/DAO để kiểm tra trong bảng TaiKhoan. Nếu đúng, hệ thống lấy thêm thông tin NhanVien và NhomQuyen, tạo Session lưu người dùng hiện tại, rồi chuyển sang MainFrame (trang chủ). Nếu sai, hiển thị thông báo lỗi và yêu cầu nhập lại.

3.2.2 Trang chủ



Hình 26 Giao diện trang chủ hệ thống

Sau khi đăng nhập, MainFrame hiển thị giao diện tổng quan gồm thanh Navigation (menu trái) để chuyển giữa các module: Sản phẩm, Bán hàng, Nhập hàng, Khách hàng, Nhân viên, Thống kê... Tùy quyền của tài khoản, một số mục có thể bị ẩn hoặc không cho thao tác.

3.2.3 Sản phẩm

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc

SoundWave Store

Sản phẩm

Bán hàng

Nhập hàng

Phiếu nhập

Phiếu xuất

Khuyến mãi

Bảo hành

Nhà cung cấp

Nhân viên

Khách hàng

Thống kê

Phân quyền

→ Đăng xuất

Tìm kiếm sản phẩm...

Thêm

Sửa

Xóa

Làm Mới

Xuất Excel

Sản Phẩm

Tất cả

STT	Mã SP	Tên Sản Phẩm	Số Lượng	Mã Loại	Mã Hãng	Mô Tả	Thời Gian Bảo H...
1	SP001	Marshall Stanmore III	300	L01	H01	Loa decor cực đẹp	12
2	SP002	Sony WH-1000XM5	280	L02	H02	Chống ồn đỉnh cao	12
3	SP003	JBL Boombox 3	299	L01	H03	Loa di động công ...	12
4	SP004	Marshall Emberton II	250	L01	H01	Loa cầm tay nhỏ g...	12
5	SP005	Marshall Middleton	300	L01	H01	Loa Bluetooth 4 lo...	12
6	SP006	Sony SRS-XE300	300	L01	H02	Loa chống nước, â...	12
7	SP007	Sony WH-CH720N	405	L02	H02	Tai nghe chống ồn...	12
8	SP008	JBL Charge 5	300	L01	H03	Loa Bluetooth kiê...	12
9	SP009	JBL PartyBox Encore	300	L01	H03	Loa kèm 2 Micro h...	12
10	SP010	JBL Flip 6	300	L01	H03	Loa di động âm th...	12
11	SP011	JBL Go 3	300	L01	H03	Loa mini chống nước	12
12	SP012	Apple AirPods Pro 2	300	L02	H03	Chống ồn chủ động	12
13	SP013	Sennheiser HD 450...	300	L02	H02	Âm thanh chi tiết, ...	24

Hình 27 Giao diện quản lý danh mục sản phẩm

3.2.3.1 Tìm kiếm/lọc: người dùng nhập từ khóa hoặc chọn loại/hãng, sự kiện trên SearchTextField/combo sẽ gọi BUS trả về danh sách đã lọc, bảng hiển thị lại.

3.2.3.2 Thêm sản phẩm:

STT	Mã sản phẩm	Tên sản phẩm	Số lượng	Mã loại	Mã Hãng	Mô tả	Thời gian Bảo hành
1							
2							
3							
4							
5							
6							
7							
8							
9							
10	SP010	JBL Flip 6	300	L01			
11	SP011	JBL Go 3	300	L01			
12	SP012	Apple AirPods Pro 2	300	L02			
13	SP013	Sennheiser HD 450...	300	L02			

Hình 28 Hộp thoại thêm thông tin sản phẩm

Bấm nút Thêm mở SuaSanPhamDialog/InputPhienBanDialog, nhập thông tin, hệ thống kiểm tra hợp lệ ở BUS, nếu đúng thì lưu qua SanPhamDAO và reload bảng.

3.2.3.3 Sửa: chọn dòng trong bảng, bấm Sửa, dialog được fill sẵn dữ liệu, sau khi chỉnh sửa và xác nhận, gọi BUS cập nhật CSDL.

3.2.3.4 Xóa: chọn sản phẩm, bấm Xóa; BUS kiểm tra ràng buộc (ví dụ đã phát sinh phiếu xuất chưa), nếu cho phép thì gọi DAO xóa/mềm, sau đó cập nhật lại bảng.

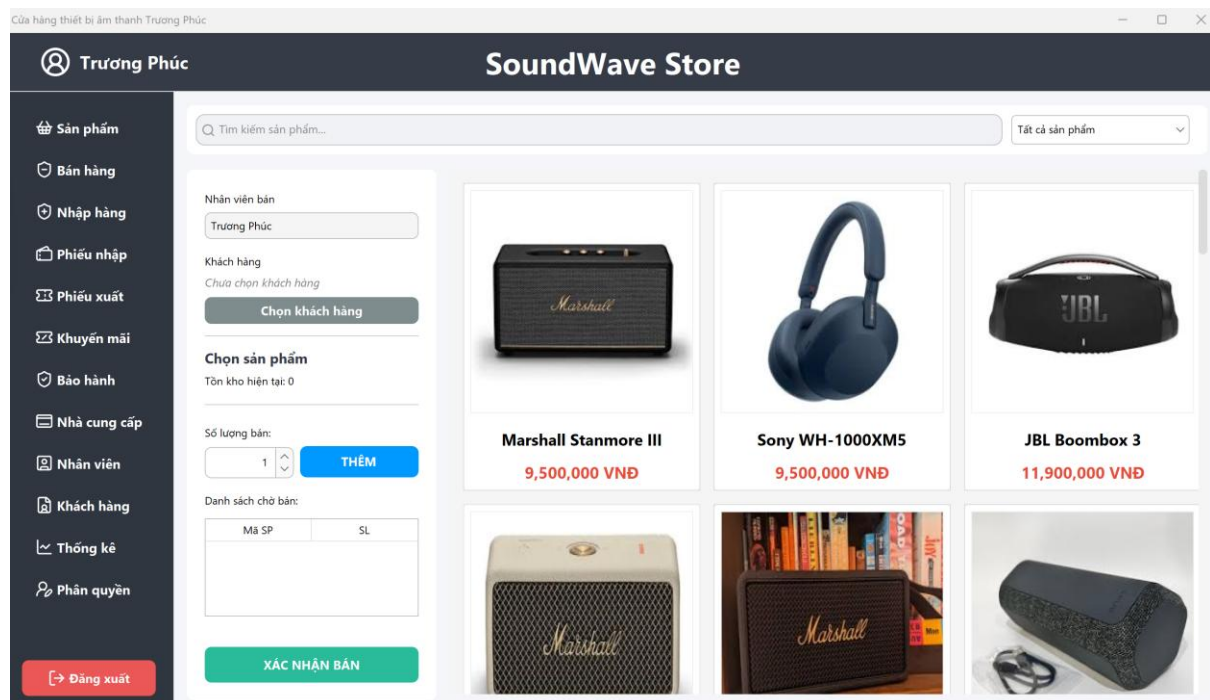
3.2.3.5 Xuất excel:

A	B	C	D	E	F	G	H	I
STT	Mã SP	Tên Sản Phẩm	Số Lượng	Mã Loại	Mã Hãng	Mô Tả	Thời Gian Bảo Hành	
1	SP001	Marshall Stanmore III	300	L01	H01	Loa decor cực đẹp	12	
2	SP002	Sony WH-1000XM5	280	L02	H02	Chống ồn đỉnh cao	12	
3	SP003	JBL Boombox 3	299	L01	H03	Loa di động công suất lớn, kháng nước IP67	12	
4	SP004	Marshall Emberton II	250	L01	H01	Loa cầm tay nhỏ gọn, pin 30h	12	
5	SP005	Marshall Middleton	300	L01	H01	Loa Bluetooth 4 loa cực mạnh	12	
6	SP006	Sony SRS-XE300	300	L01	H02	Loa chống nước, âm thanh rộng	12	
7	SP007	Sony WH-CH720N	405	L02	H02	Tai nghe chống ồn giá rẻ	12	
8	SP008	JBL Charge 5	300	L01	H03	Loa Bluetooth kiêm sạc dự phòng	12	
9	SP009	JBL PartyBox Encore	300	L01	H03	Loa kèm 2 Micro hát karaoke	12	
10	SP010	JBL Flip 6	300	L01	H03	Loa di động âm thanh 2 đường tiếng	12	
11	SP011	JBL Go 3	300	L01	H03	Loa mini chống nước	12	
12	SP012	Apple AirPods Pro 2	300	L02	H03	Chống ồn chủ động	12	
13	SP013	Sennheiser HD 450BT	300	L02	H02	Âm thanh chi tiết, pin 30h	24	
14	SP014	Bose QuietComfort 45	300	L02	H03	Khử tiếng ồn đỉnh cao	12	
15	SP015	JBL Tune 510BT	300	L02	H03	Bass mạnh, giá sinh viên	12	
16	SP016	Sony WH-1000XM4	300	L02	H02	Bản nâng cấp huyền thoại	12	
17	SP017	Marshall Major IV	300	L02	H01	Pin 80h, sạc không dây	12	
18	SP018	AirPods Max	300	L02	H03	Tai nghe Over-ear cao cấp	12	
19	SP019	Micro Shure SM58	300	L03	H03	Micro vocal chuyên nghiệp	12	
20	SP020	Cáp Audio 3.5mm Ugreen	300	L03	H02	Dây bọc dù chống nhiễu	6	
21	SP021	Giá treo tai nghe Gỗ	300	L03	H01	Decor gỗ Walnut sang trọng	0	
22	SP022	Bao da Marshall Willen	300	L03	H01	Bao bảo vệ silicon	3	
23	SP023	Micro JBL PBM100	300	L03	H03	Micro karaoke có dây	12	
24	SP024	Cáp sạc Type-C bọc thép	300	L03	H02	Sạc siêu bền cho loa	6	
25	SP025	Bộ vệ sinh tai nghe 3 in 1	300	L03	H02	Dạng bút vệ sinh tiện lợi	0	
26	SP026	Jack chuyển 6.35mm sang 3.5mm	300	L03	H02	Mạ vàng cao cấp	6	
27	SP027	Túi đựng loa JBL Charge	300	L03	H03	Chống sốc EVA cao cấp	3	
28	SP028	Sony x200	0	L01	H02		12	

Hình 29 Xuất dữ liệu sản phẩm ra file Excel

Bấm nút xuất, component XuatExcel đọc dữ liệu hiện có trong bảng và ghi ra file Excel theo định dạng cố định.

3.2.4 Bán hàng



Hình 30 Giao diện bán hàng(Tạo giỏ hàng)

3.2.4.1 Luồng xử lý:

1. Chọn sản phẩm và tạo giỏ hàng

Ở BanHangPanel, hệ thống tải danh sách sản phẩm còn kinh doanh từ SanPhamBUS và hiển thị dưới dạng lưới ProductGrid (ảnh, tên, giá). Khi nhân viên bấm chọn một sản phẩm, chương trình kiểm tra tồn kho hiện tại; nếu còn hàng sẽ thêm sản phẩm đó vào danh sách “giỏ hàng” ở bên trái, kèm số lượng mặc định là 1. Nhân viên có thể tăng/giảm số lượng, xóa mặt hàng khỏi giỏ; mỗi thao tác đều tính lại tổng tiền tạm tính.

Nhân viên bán

Trương Phúc

Khách hàng

Chưa chọn khách hàng

Chọn khách hàng

Chưa chọn sản phẩm

Tồn kho hiện tại: 0

Số lượng bán:

1

THÊM

Danh sách chờ bán:

Mã SP	SL
PB001	1
PB002	15

XÁC NHẬN BÁN



Marshall Stanmore III

9,500,000 VNĐ



Sony WH-1000XM5

9,500,000 VNĐ





Hình 31 Chọn sản phẩm cần bán

2. Chọn khách hàng

Nhân viên có thể bấm nút chọn khách, mở ChonKhachHangDialog: dialog này hiển thị KháchHangTable

Chọn khách hàng

DANH SÁCH KHÁCH HÀNG

Mã KH	Tên Khách Hàng	Số Điện Thoại	Địa Chỉ
KH001	Nguyễn Khách	0912345678	TP.HCM
KH002	Trần VIP	0987654321	Cần Thơ
KH003	Phương	0929910893	Q7, TP.HCM

THÊM MỚI

CHỌN

Hình 32 Hộp thoại chọn khách hàng hoặc thêm khách hàng mới

3. Xác nhận bán hàng

Khi giỏ hàng, khách hàng đã sẵn sàng, nhân viên bấm “Xác nhận bán”, hệ thống mở XacNhanBanHangDialog để kiểm tra lần cuối (danh sách sản phẩm, số lượng, giá, tổng thanh toán)

Xác nhận hóa đơn bán hàng

X

KIỂM TRA THÔNG TIN HÓA ĐƠN

STT	Mã SP	Tên Sản Phẩm	SL	Đơn Giá	Thành Tiền
1	PB001	Marshall Stanmore III	1	9,500,000	9,500,000
2	PB002	Sony WH-1000XM5	15	9,500,000	142,500,000

Tổng số lượng: 16

TỔNG THANH TOÁN: 152,000,000 VNĐ

CHỌN KHUYẾN MÃI

HỦY BỎ

XÁC NHẬN & XUẤT HÓA ĐƠN

Hình 33 Hộp thoại xác nhận bán hàng

4. Chọn khuyến mãi (nếu có)

Nếu có chương trình giảm giá, nhân viên mở ChonKhuyenMaiDialog, hệ thống chỉ hiển thị các khuyến mãi đang còn hiệu lực (dựa trên ngày bắt đầu/kết thúc, trạng thái) từ KhuyenMaiBUS. Khi chọn, mức giảm sẽ được áp dụng lên tổng tiền, hiển thị rõ ràng để tránh nhầm lẫn.

Chọn chương trình khuyến mãi

X

DANH SÁCH KHUYẾN MÃI ĐANG ÁP DỤNG

Mã KM	Tên Chương Trình	% Giảm	Ngày Kết Thúc
KM001	Khai trương	10.0%	2030-12-31
KM005	Tri ân khách hàng VIP	25.0%	2026-12-31
KM008	Mua loa tặng phụ kiện	12.0%	2026-03-31

ĐÓNG

XÁC NHẬN CHỌN

Hình 34 Hộp thoại chọn khuyến mãi còn hạn

5. Ghi dữ liệu và xuất hóa đơn

Tạo bản ghi `PhieuXuat` (ngày lập, nhân viên, khách hàng, mã khuyến mãi, tổng tiền...).

Sinh các dòng `CTPhieuXuat` tương ứng từng sản phẩm trong giỏ (mã phiếu, mã sản phẩm, số lượng, đơn giá, thành tiền).

Cập nhật tồn kho: trừ số lượng trong bảng `SanPham` hoặc `PhienBanSanPham`.

Có thể gọi chức năng in hóa đơn PDF (`XuatPDFHoaDonXuat`) để lưu/trao cho khách.

PHIẾU BÁN HÀNG

Mã phiếu: PX007

Nhân viên lập: NV001

Ngày lập: Fri Mar 13 14:28:24 GMT+07:00 2026

STT	Tên Sản Phẩm	SL	Đơn Giá	Tổng tiền	Thành Tiền
1	Marshall Stanmore III	1	9,500,000	9,500,000	8,550,000
2	Sony WH-1000XM5	15	9,500,000	142,500,000	128,250,000
TỔNG TIỀN THANH TOÁN					136,800,000 VNĐ

Hình 35 Hóa đơn xuất phiếu bán hàng(File PDF)

3.2.5 Nhập hàng:

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc

SoundWave Store

Sản phẩm

Bán hàng

Nhập hàng

Phiếu nhập

Phiếu xuất

Khuyến mãi

Bảo hành

Nhà cung cấp

Nhân viên

Khách hàng

Thống kê

Phân quyền

Q Tìm kiếm tên SP, mã SP...

Nhập Excel

Tất cả sản phẩm

Nhân viên nhập:
Trương Phúc

Nhà cung cấp:
Tất cả nhà cung cấp

Chọn sản phẩm
Tên kho hiện tại: 0

Số lượng nhập:
1

THÊM

Danh sách chờ nhập:

Mã SP

SL

XÁC NHẬN NHẬP

STT	Mã Phiên Bản	Tên Sản Phẩm	Màu Sắc	Giá Nhập	Số Lượng Tôn
1	PB001	Marshall Stanmore III	Kem (Cream)	7,000,000	300
2	PB002	Sony WH-1000XM5	Đen (Black)	7,000,000	280
3	PB003	JBL Boombox 3	Đen (Black)	8,500,000	299
4	PB004	Marshall Emberton II	Đen Brass	3,200,000	250
5	PB005	Marshall Middleton	Đen (Black)	6,500,000	300
6	PB006	Sony SRS-XE300	Xám (Grey)	2,800,000	300
7	PB007	Sony WH-CH720N	Xanh (Blue)	1,800,000	405
8	PB008	JBL Charge 5	Đen (Black)	3,000,000	300
9	PB009	JBL PartyBox Encore	Đen (Black)	5,800,000	300
10	PB010	JBL Flip 6	Đỏ (Red)	2,200,000	300
11	PB011	JBL Go 3	Xanh quân đội	800,000	300
12	PB012	Apple AirPods Pro 2	Trắng	4,500,000	300
13	PB013	Sennheiser HD 450BT	Đen	2,800,000	300
14	PB014	Bose QuietComfort 45	Trắng	5,500,000	300
15	PB015	JBL Tune 510BT	Hồng	700,000	300
16	PB016	Sony WH-1000XM4	Bạc	5,000,000	300

Hình 36 Chọn sản phẩm để nhập hàng vào kho

3.2.5.1 Luồng xử lý nhập hàng:

1. Chọn sản phẩm cần nhập và khai báo số lượng, giá nhập

Hệ thống hiển thị bảng/dialog chọn sản phẩm, cho phép tìm theo tên, mã, loại. Với mỗi sản phẩm được thêm vào phiếu, nhân viên nhập số lượng, đơn giá nhập, có thể ghi chú thêm (lô hàng, bảo hành NCC...). Các dòng hàng được lưu tạm trong danh sách chi tiết phiếu nhập và tính ra tổng tiền dự kiến. Ở bước này, BUS có thể kiểm tra các điều kiện cơ bản như số lượng > 0, giá nhập không âm...

Nhân viên nhập:

Trương Phúc

Nhà cung cấp:

Tất cả nhà cung cấp

Chưa chọn sản phẩm

Tồn kho hiện tại: 0

Số lượng nhập:

1

THÊM

Danh sách chờ nhập:

Mã SP	SL
PB002	50
PB005	50
PB010	50

XÁC NHẬN NHẬP

STT	Mã Phiên Bản	Tên Sản Phẩm	Màu Sắc	Giá Nhập	Số Lượng Tồn
1	PB001	Marshall Stanmore III	Kem (Cream)	7,000,000	299
2	PB002	Sony WH-1000XM5	Đen (Black)	7,000,000	265
3	PB003	JBL Boombox 3	Đen (Black)	8,500,000	299
4	PB004	Marshall Emberton II	Đen Brass	3,200,000	250
5	PB005	Marshall Middleton	Đen (Black)	6,500,000	300
6	PB006	Sony SRS-XE300	Xám (Grey)	2,800,000	300
7	PB007	Sony WH-CH720N	Xanh (Blue)	1,800,000	405
8	PB008	JBL Charge 5	Đen (Black)	3,000,000	300
9	PB009	JBL PartyBox Encore	Đen (Black)	5,800,000	300
10	PB010	JBL Flip 6	Đỏ (Red)	2,200,000	300
11	PB011	JBL Go 3	Xanh quân đội	800,000	300
12	PB012	Apple AirPods Pro 2	Trắng	4,500,000	300
13	PB013	Sennheiser HD 450BT	Đen	2,800,000	300
14	PB014	Bose QuietComfort 45	Trắng	5,500,000	300
15	PB015	JBL Tune 510BT	Hồng	700,000	300
16	PB016	Sony WH-1000XM4	Bạc	5,000,000	300
17	PB017	Marshall Major IV	Nâu da	3,000,000	300
18	PB018	AirPods Max	Xanh Sky	10,000,000	300
19	PB019	Micro Shure SM58	Xám	2,100,000	300

Hình 37 Hộp thoại xác nhận nhập hàng vào kho

2. Xác nhận nhập hàng

Khi admin/ nhân viên nhập hàng đã sẵn sàng, nhân viên bấm “Xác nhận nhập”, hệ thống mở XacNhanNhapHangDialog để kiểm tra lần cuối

KIỂM TRA THÔNG TIN NHẬP HÀNG

STT	Mã Phiếu...	Tên Sản Phẩm	SL	Đơn Giá	Thành Tiền
1	PB002	Sony WH-1000XM5	50	7,000,000	350,000,000
2	PB005	Marshall Middleton	50	6,500,000	325,000,000
3	PB010	JBL Flip 6	50	2,200,000	110,000,000

Tổng số lượng: 150**TỔNG TIỀN THANH TOÁN: 785,000,000 VNĐ**

HỦY BỎ

XÁC NHẬN & XUẤT HÓA ĐƠN

Hình 38 Hộp thoại xác nhận nhập hàng vào kho

Tạo bản ghi PhieuNhap (mã phiếu, NCC, nhân viên lập, ngày lập, tổng tiền...).

Thêm các bản ghi CTPhieuNhap cho từng sản phẩm (mã phiếu, mã sản phẩm, số lượng nhập, đơn giá).

Cập nhật tồn kho: cộng thêm số lượng nhập vào kho hiện tại của từng sản phẩm/phiên bản sản phẩm.

(Tùy thiết kế) Cập nhật giá vốn trung bình hoặc lưu lịch sử giá nhập để phục vụ thống kê.

Sau khi hoàn tất, giao diện reload danh sách phiếu nhập và tồn kho mới;

Có thể hỗ trợ nhập hàng bằng file Excel

Mã Phiên Bản	Tên Sản Phẩm	Số Lượng	Đơn Giá
PB001	Marshall Stanmore III	100	7000000
PB002	Sony WH-1000XM5	100	7000000
PB003	JBL Boombox 3	100	8500000
PB004	Marshall Emberton II	100	3200000
PB005	Marshall Middleton	100	6500000
PB006	Sony SRS-XE300	100	2800000
PB007	Sony WH-CH720N	100	1800000
PB008	JBL Charge 5	100	3000000
PB009	JBL PartyBox Encore	100	5800000
PB010	JBL Flip 6	100	2200000
PB011	JBL Go 3	100	800000
PB012	Apple AirPods Pro 2	100	4500000
PB013	Sennheiser HD 450BT	100	2800000
PB014	Bose QuietComfort 45	100	5500000
PB015	JBL Tune 510BT	100	700000

Hình 39 Nhập hàng hàng loạt từ file Excel

Có thể gọi chức năng in hóa đơn PDF (XuấtPDFHoaDonNhap) để lưu

PHIẾU NHẬP HÀNG

Mã phiếu: PN007

Nhân viên lập: NV001

Ngày lập: Fri Mar 13 14:40:19 GMT+07:00 2026

STT	Tên Sản Phẩm	SL	Đơn Giá	Thành Tiền
1	Sony WH-1000XM5	50	7,000,000	350,000,000
2	Marshall Middleton	50	6,500,000	325,000,000
3	JBL Flip 6	50	2,200,000	110,000,000
TỔNG TIỀN THANH TOÁN				785,000,000 VNĐ

Hình 40 Xuất hóa đơn nhập ra file PDF

3.2.6 Phiếu nhập

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc

SoundWave Store

Sản phẩm

Bán hàng

Nhập hàng

Phiếu nhập

Phiếu xuất

Khuyến mãi

Bảo hành

Nhà cung cấp

Nhân viên

Khách hàng

Thống kê

Phân quyền

[-> Đăng xuất]

MẮC ĐỊNH

Q Tìm phiếu nhập hàng ...

Xuất EXCEL

Từ ngày:

01/01/2000

Đến ngày:

12/03/2026

Số tiền từ (VND):

Đến số tiền (VND):

STT	Mã HDN	Ngày Lập	Mã NV	Tổng Tiền
1	PN006	11/03/2026 07:21	NV001	8,390,000,000
2	PN005	11/03/2026 07:19	NV001	8,390,000,000
3	PN004	11/03/2026 07:16	NV001	880,000,000
4	PN003	10/03/2026 08:45	NV001	160,000,000
5	PN002	10/03/2026 08:36	NV001	8,390,000,000
6	PN001	10/03/2026 08:33	NV001	21,000,000

Hình 41 Giao diện quản lý danh sách phiếu nhập

Phiếu nhập là chứng từ ghi nhận toàn bộ thông tin của một lần nhập hàng từ nhà cung cấp vào kho. Mỗi phiếu nhập thường chứa các dữ liệu chính như: nhà cung cấp, nhân viên lập phiếu, ngày lập, tổng tiền, ghi chú và danh sách chi tiết các mặt hàng được nhập (mã sản phẩm, tên, số lượng, đơn giá, thành tiền). Khi lưu phiếu nhập, hệ thống sẽ tự động cập nhật lại tồn kho (cộng số lượng vào kho), đồng thời lưu lịch sử nhập để sau này có thể tra cứu nguồn hàng, đối chiếu với nhà cung cấp và thống kê chi phí nhập. Nhờ quản lý tập trung qua phiếu nhập, cửa hàng dễ dàng kiểm soát dòng hàng đi vào, phát hiện chênh lệch số lượng giữa thực tế và hóa đơn NCC, cũng như tính toán giá vốn, hiệu quả hợp tác với từng nhà cung cấp.

3.2.7 Phiếu xuất

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc

SoundWave Store

Sản phẩm

Bán hàng

Nhập hàng

Phiếu nhập

Phiếu xuất

Khuyến mãi

Bảo hành

Nhà cung cấp

Nhân viên

Khách hàng

Thống kê

Phân quyền

Q Tìm phiếu xuất hàng...

Xuất EXCEL

Từ ngày:
01/01/2000

Đến ngày:
12/03/2026

Số tiền từ (VND):

Đến số tiền (VND):

MẮC ĐỊNH

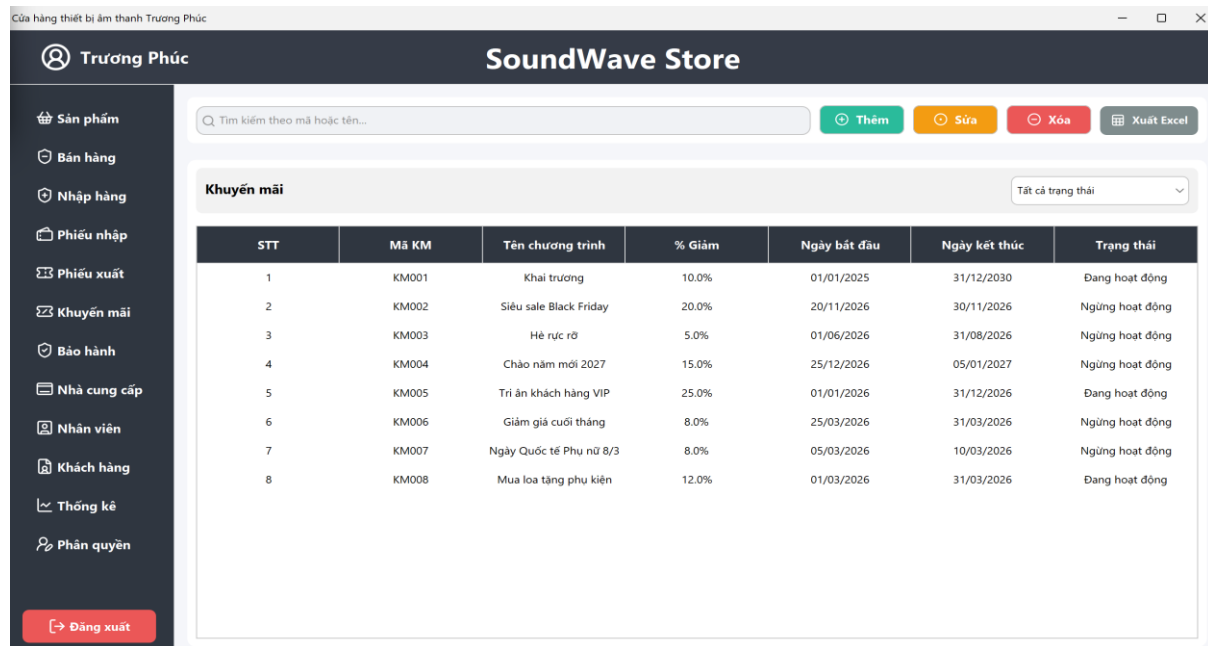
STT	Mã HĐX	Ngày Bán	Mã NV	Khách Hàng	Tổng Tiền
1	PX001	10/03/2026 08:33	NV001	KH001	23,085,000 VNĐ
2	PX002	10/03/2026 08:45	NV001	KH003	490,500,000 VNĐ
3	PX003	10/03/2026 10:57	NV001	KH003	95,000,000 VNĐ
4	PX004	10/03/2026 11:07	NV001	KH001	11,900,000 VNĐ
5	PX005	11/03/2026 06:57	NV006	KH003	95,000,000 VNĐ
6	PX006	11/03/2026 07:12	NV001	KH003	641,250,000 VNĐ

[-> Đăng xuất]

Hình 42 Giao diện quản lý danh sách phiếu xuất

Phiếu xuất là chứng từ ghi nhận việc xuất hàng ra khỏi kho, chủ yếu dưới dạng bán cho khách hàng. Một phiếu xuất thường bao gồm: thông tin khách hàng, nhân viên bán hàng, ngày lập, chương trình khuyến mãi áp dụng (nếu có), tổng tiền thanh toán, cùng danh sách chi tiết các sản phẩm được bán (mã sản phẩm, số lượng, đơn giá bán, thành tiền, mức giảm giá). Khi xác nhận phiếu xuất, hệ thống tự động trừ tồn kho tương ứng, cập nhật doanh thu, có thể sinh hóa đơn cho khách (PDF/Excel) và lưu lại để phục vụ đổi trả, bảo hành về sau. Việc quản lý bán hàng thông qua phiếu xuất giúp cửa hàng nắm rõ lịch sử giao dịch, dễ thống kê doanh thu theo thời gian, theo nhân viên, theo sản phẩm, đồng thời đảm bảo dữ liệu kho luôn khớp với thực tế.

3.2.8 Khuyến mãi



STT	Mã KM	Tên chương trình	% Giảm	Ngày bắt đầu	Ngày kết thúc	Trạng thái
1	KM001	Khai trương	10.0%	01/01/2025	31/12/2030	Đang hoạt động
2	KM002	Siêu sale Black Friday	20.0%	20/11/2026	30/11/2026	Ngừng hoạt động
3	KM003	Hè rực rỡ	5.0%	01/06/2026	31/08/2026	Ngừng hoạt động
4	KM004	Chào năm mới 2027	15.0%	25/12/2026	05/01/2027	Ngừng hoạt động
5	KM005	Tri ân khách hàng VIP	25.0%	01/01/2026	31/12/2026	Đang hoạt động
6	KM006	Giảm giá cuối tháng	8.0%	25/03/2026	31/03/2026	Ngừng hoạt động
7	KM007	Ngày Quốc tế Phụ nữ 8/3	8.0%	05/03/2026	10/03/2026	Ngừng hoạt động
8	KM008	Mua loa tặng phụ kiện	12.0%	01/03/2026	31/03/2026	Đang hoạt động

Hình 43 Giao diện quản lý các chương trình khuyến mãi

Khuyến mãi là module dùng để quản lý các chương trình giảm giá áp dụng cho đơn hàng hoặc sản phẩm. Mỗi khuyến mãi có các thông tin chính như: tên chương trình, loại giảm (theo %, theo số tiền), điều kiện áp dụng, ngày bắt đầu – kết thúc, trạng thái (đang áp dụng / hết hạn). Nhân viên quản trị có thể thêm khuyến mãi mới, chỉnh sửa nội dung (mức giảm, thời gian, điều kiện) hoặc tạm ngưng/xóa các chương trình cũ. Khi bán hàng, hệ thống chỉ cho phép chọn những khuyến mãi đang hiệu lực; sau khi áp dụng, giá trị giảm sẽ được tính vào phiếu xuất và lưu lại để sau này có thể thống kê hiệu quả từng chương trình.

3.2.9 Bảo hành

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc		SoundWave Store			
Sản phẩm Bán hàng Nhập hàng Phiếu nhập Phiếu xuất Khuyến mãi Bảo hành Nhà cung cấp Nhân viên Khách hàng Thống kê Phân quyền		Q Tìm kiếm thiết bị bảo hành (IMEI, Mã BH)... ThêmXóaXuất Excel			
Bảo hành Tất cả trạng thái					
STT	Mã bảo hành	Mã Imei	Mã phiếu xuất	Ngày bắt đầu	Ngày kết thúc
1	BH001	111222333	PX001	2026-03-10	2027-03-10
2	BH002	444555666	PX001	2026-01-10	2027-01-10
3	BH003	777888999	PX001	2026-02-05	2027-02-05
4	BH004	101870999	PX002	2026-03-10	2027-03-10
5	BH005	102031437	PX002	2026-03-10	2027-03-10
6	BH006	105604267	PX002	2026-03-10	2027-03-10
7	BH007	107216635	PX002	2026-03-10	2027-03-10
8	BH008	113747056	PX002	2026-03-10	2027-03-10
9	BH009	128114052	PX002	2026-03-10	2027-03-10
10	BH010	131043957	PX002	2026-03-10	2027-03-10
11	BH011	137046039	PX002	2026-03-10	2027-03-10
12	BH012	137876167	PX002	2026-03-10	2027-03-10
13	BH013	142897347	PX002	2026-03-10	2027-03-10

Đăng xuất

Hình 44 Giao diện quản lý thông tin bảo hành thiết bị

Bảo hành dùng để ghi nhận và theo dõi các yêu cầu bảo hành sau khi sản phẩm đã bán. Thông tin chính gồm: sản phẩm, khách hàng, phiếu xuất liên quan, ngày nhận bảo hành, mô tả lỗi, trạng thái xử lý, ngày trả hàng... Khi tiếp nhận yêu cầu, nhân viên tạo phiếu bảo hành mới; trong quá trình xử lý có thể cập nhật trạng thái (đang kiểm tra, đã sửa xong, đổi mới...), ghi chú những linh kiện thay thế hoặc quyết định đổi/trả. Dữ liệu bảo hành giúp cửa hàng theo dõi chất lượng sản phẩm, làm căn cứ làm việc với nhà cung cấp và hỗ trợ khách hàng minh bạch hơn.

3.2.10 Nhà cung cấp

Cửa hàng thiết bị âm thanh Trương Phúc				
Trương Phúc				
SoundWave Store				
Q Tìm kiếm nhà cung cấp ...				
Thêm Sửa Xóa Xuất Excel				
Nhà cung cấp				
Tất cả				
STT	Mã Nhà cung cấp	Tên Nhà cung cấp	Địa chỉ	SĐT
1	NCC001	Marshall VN Dist	Q1, TP.HCM	0283333089
2	NCC003	Bose	Hà Nội	0991299099
3	NCC002	B&O	Q7, TP.HCM	0961254087

Hình 45 Giao diện quản lý thông tin nhà cung cấp

Nhà cung cấp là nơi quản lý thông tin các đơn vị cung cấp thiết bị âm thanh cho cửa hàng. Mỗi NCC có các trường như tên, địa chỉ, số điện thoại, email, người liên hệ, tình trạng hợp tác. Hệ thống cho phép thêm NCC mới, sửa thông tin khi có thay đổi, hoặc ngừng sử dụng với những NCC không còn hợp tác. Khi lập phiếu nhập, nhân viên chọn nhà cung cấp từ danh sách này; nhờ đó có thể thống kê tổng giá trị nhập theo NCC, đánh giá nhà cung cấp nào ổn định, giá cả tốt.

3.2.11 Nhân viên

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc

SoundWave Store

Sản phẩm

Bán hàng

Nhập hàng

Phiếu nhập

Phiếu xuất

Khuyến mãi

Bảo hành

Nhà cung cấp

Nhân viên

Khách hàng

Thống kê

Phân quyền

[-> Đăng xuất]

Q Tìm kiếm nhân viên ...

Thêm

Sửa

Xóa

Xuất Excel

Nhân viên

Tên mặc định

Tất cả chức vụ

Lương mặc định

STT	Mã NV	Tên NV	SĐT	Địa chỉ	Chức vụ	Email	Lương
1	NV001	Trương Phúc	0909123456	Đà Nẵng	Quản lý	phuc@sw.com	20,000,000
2	NV002	Lê Văn Nam	0909123457	Hà Nội	Nhân viên bán hàng	nam@sw.com	10,000,000
3	NV003	Nguyễn Văn Kho	0909999888	Hải Phòng	Nhân viên kho	kho@sw.com	12,000,000
4	NV006	Minh Khuyển	0979920899	Q8, TP.HCM	Nhân viên bán hàng	khuyen@gmail.com	10,000,000

Hình 46 Giao diện quản lý thông tin nhân viên

Nhân viên quản lý hồ sơ từng người làm việc tại cửa hàng: họ tên, ngày sinh, giới tính, liên hệ, chức vụ, lương cơ bản (nếu cần), tài khoản đăng nhập và nhóm quyền. Phần mềm cho phép thêm nhân viên mới, chỉnh sửa thông tin cá nhân, phân quyền theo vai trò (thu ngân, quản lý, kho, admin...), và tạm khóa/xóa tài khoản khi nhân viên nghỉ việc. Dựa vào dữ liệu này, hệ thống gán nhân viên lập phiếu nhập, phiếu xuất, từ đó có thể thống kê doanh số theo từng người và kiểm soát trách nhiệm.

3.2.12 Khách hàng

Cửa hàng thiết bị âm thanh Trương Phúc

Trương Phúc

SoundWave Store

Sản phẩm

Bán hàng

Nhập hàng

Phiếu nhập

Phiếu xuất

Khuyến mãi

Bảo hành

Nhà cung cấp

Nhân viên

Khách hàng

Thống kê

Phân quyền

Q Tìm kiếm khách hàng ...

Thêm

Sửa

Xóa

Xuất Excel

Khách hàng

Tên mặc định

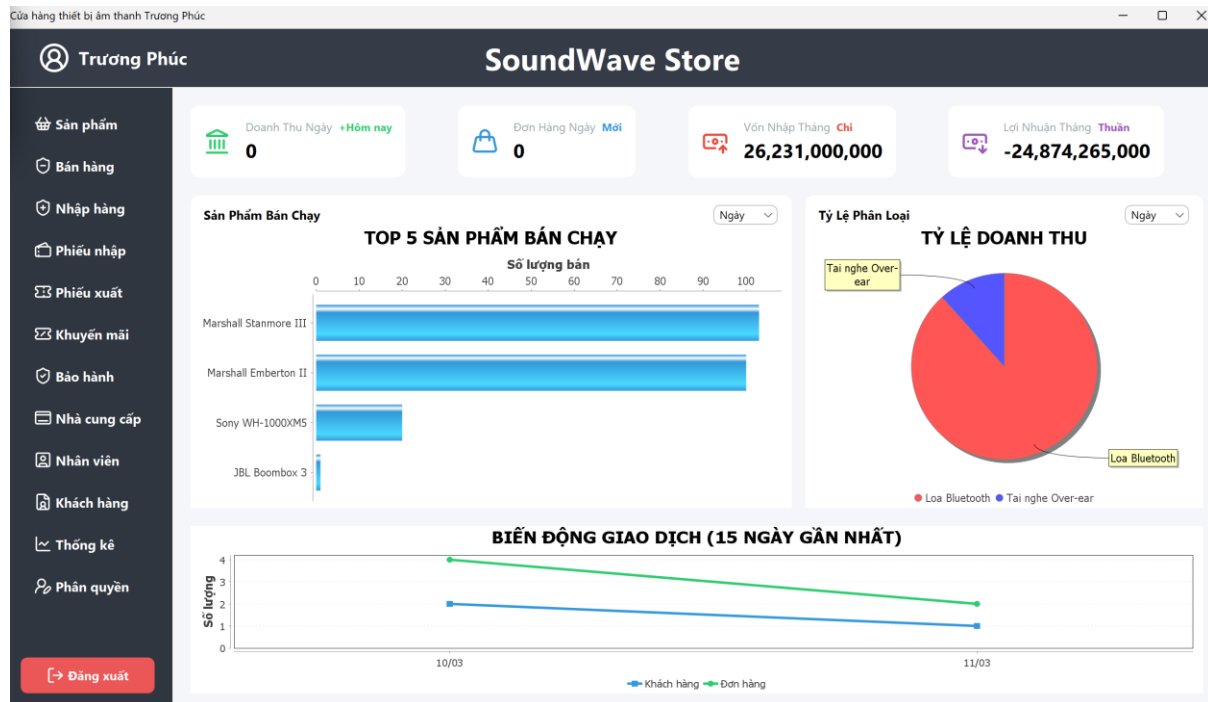
STT	Mã KH	Tên KH	SĐT	Địa chỉ
1	KH001	Nguyễn Khách	0912345678	TP.HCM
2	KH002	Trần VIP	0987654321	Cần Thơ
3	KH003	Phương	0929910893	Q7, TP.HCM

→ Đăng xuất

Hình 47 Giao diện thông tin khách hàng

Khách hàng là module lưu trữ thông tin người mua: họ tên, số điện thoại, địa chỉ, email, loại khách (thân thiết, lẻ...), ghi chú. Khi bán hàng, nhân viên có thể tìm nhanh khách theo số điện thoại/tên để gắn vào phiếu xuất, giúp lưu lại lịch sử mua sắm. Việc thêm/sửa/xóa khách hàng tương tự sản phẩm: có form nhập, bảng danh sách, tính năng tìm kiếm. Dữ liệu khách hàng được dùng để tra cứu khi cần bảo hành/đổi trả, đồng thời hỗ trợ phân tích hành vi mua sắm, triển khai chương trình chăm sóc hoặc khuyến mãi dành riêng cho từng nhóm khách.

3.2.13 Thống kê



Hình 48 Giao diện bảng điều khiển, thống kê tổng quan

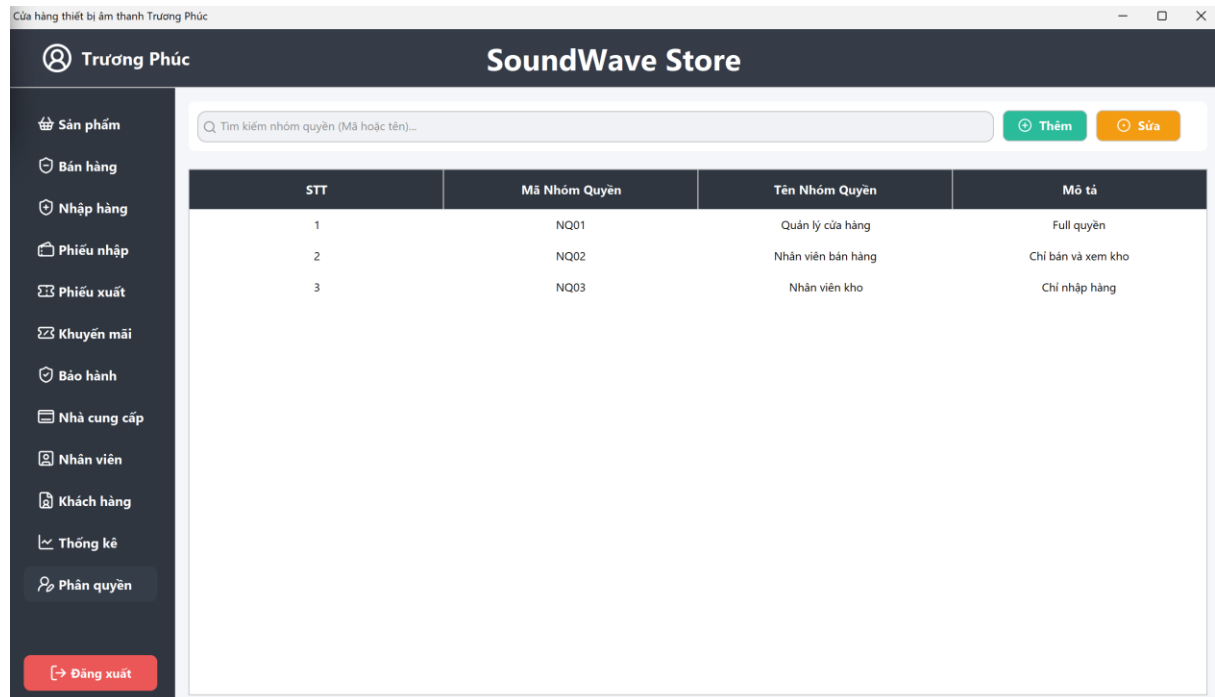
Chức năng thống kê dùng để tổng hợp và trực quan hóa dữ liệu bán hàng, nhập hàng và kho, giúp người quản lý nắm tình hình kinh doanh theo nhiều góc nhìn.

PhieuXuat, CTPhieuXuat, PhieuNhap, SanPham, KhachHang... và được xử lý ở tầng ThongKeBUS, sau đó hiển thị bằng các bảng và biểu đồ trong các panel như KpiCardsPanel, ProductChartPanel. Hệ thống có thể thống kê doanh thu theo ngày/tháng/năm, so sánh giữa các khoảng thời gian, thống kê sản phẩm bán chạy, nhóm sản phẩm có doanh số cao/thấp, hay tổng hợp số lượng đơn hàng, số khách mới trong kỳ.

Về luồng xử lý, khi người dùng chọn khoảng thời gian cần xem (ví dụ tháng 1/2026), giao diện gửi yêu cầu tới ThongKeBUS. Tầng nghiệp vụ thực hiện các truy vấn tổng hợp (SUM, COUNT, GROUP BY...) trên CSDL để tính doanh thu, số lượng sản phẩm bán, lợi nhuận ước tính, sau đó trả về các DTO thống kê

(ThongKeDoanhThu, ThongKeSanPham, ThongKeKhachHang...). Kết quả được hiển thị dạng thẻ KPI (tổng doanh thu, số đơn, số khách), bảng (top sản phẩm) và biểu đồ (cột/đường/tròn) giúp người quản lý dễ nhận ra xu hướng. Nhờ đó, cửa hàng có cơ sở quyết định nhập thêm mặt hàng nào, cắt giảm mặt hàng nào, điều chỉnh giá hoặc tung chương trình khuyến mãi phù hợp.

3.2.14 Phân quyền



Hình 49 Giao diện cài đặt và phân quyền người dùng

Phân quyền là chức năng đảm bảo mỗi tài khoản chỉ được thao tác trong phạm vi công việc của mình, tránh lạm dụng hoặc sai sót ngoài ý muốn. Hệ thống sử dụng các thực thể chính:

TaiKhoan: thông tin đăng nhập (tên đăng nhập, mật khẩu mã hóa, trạng thái...).

NhomQuyền: nhóm quyền/vai trò (Admin, Quản lý, Nhân viên bán hàng, Nhân viên kho...).

ChucNang: danh sách các chức năng cụ thể trong hệ thống (quản lý sản phẩm, xem thống kê, chỉnh sửa khuyến mãi, xóa phiếu...).

Quan hệ giữa NhomQuyền và ChucNang là n-n, được cài đặt qua bảng trung gian ChiTietQuyền. Ở màn hình phân quyền (PhanQuyềnToolBar, các dialog liên quan), người quản trị có thể: tạo nhóm quyền mới, gán cho nhóm đó những chức năng được phép sử dụng (thêm/sửa/xóa/xem ở từng module), hoặc chỉnh sửa nhóm quyền hiện có. Khi tạo tài khoản cho nhân viên, hệ thống gán mỗi TaiKhoan vào một NhomQuyền.

Trong quá trình chạy, sau khi đăng nhập, thông tin nhóm quyền và các chức năng được phép sẽ được lưu trong Session. Mỗi khi người dùng mở một màn hình hay thực hiện hành động nhạy cảm (xóa dữ liệu, xem thống kê, truy cập module quản trị...), hệ thống kiểm tra quyền tương ứng trong ChiTietQuyền. Nếu không đủ quyền, nút chức năng sẽ

bị ẩn hoặc hiển thị thông báo không được phép. Cách thiết kế này giúp tách riêng dữ liệu phân quyền khỏi code giao diện, dễ dàng thêm quyền mới, chỉnh sửa quyền cho cả nhóm mà không phải sửa trực tiếp từng màn hình, đồng thời đảm bảo an toàn dữ liệu và tính kỷ luật trong vận hành.

3.2.15 Đổi trả (Chức năng thêm)

Cửa hàng thiết bị âm thanh Trương Phúc

SoundWave Store							
Sân phẩm		Tìm kiếm mã DT, mã PX, IMEI, khách hàng...		Thêm Sửa Xóa		Xuất Excel	
Danh sách đổi trả							
Mã Đổi Trả	Mã PX	Khách Hàng	Ngày Mua	Ngày Trả	Hạn Cuối	Mã IMEI	Lý Do
DT001	PX001	Nguyễn Khách	16/03/2026	20/03/2026	15/04/2026	111222333	Loa bị rè bass
DT002	PX002	Trần VIP	16/03/2026	16/03/2026	15/04/2026	112990479	Lỗi loa
DT003	PX003	Trần VIP	16/03/2026	16/03/2026	15/04/2026	138367587	Nút vỡ
DT004	PX004	Nguyễn Khách	16/03/2026	16/03/2026	15/04/2026	221465732	Thay đổi lựa chọn
DT005	PX005	Trần VIP	16/03/2026	16/03/2026	15/04/2026	200610158	Loa rè
DT006	PX006	Trần VIP	16/03/2026	16/03/2026	15/04/2026	115713747	Hư vật
DT007	PX007	Trần VIP	16/03/2026	16/03/2026	15/04/2026	103278078	Loa rè
DT008	PX008	Trần VIP	16/03/2026	16/03/2026	15/04/2026	693622168	Loa rè
DT009	PX009	Trần VIP	17/03/2026	17/03/2026	16/04/2026	698409082	Thay đổi nhu cầu
DT010	PX010	Trần VIP	17/03/2026	17/03/2026	16/04/2026	333589071	Thay đổi nhu cầu
DT011	PX011	Nguyễn Khách	17/03/2026	17/03/2026	16/04/2026	285378758	Loa rè
DT012	PX012	Trần VIP	17/03/2026	17/03/2026	16/04/2026	296631874	Loa rè

Hình 50 Giao diện quản lý yêu cầu đổi trả sản phẩm

Chức năng đổi trả dùng để ghi nhận và xử lý các yêu cầu đổi/trả phát sinh sau khi khách mua hàng, giúp cửa hàng quản lý minh bạch lịch sử hậu mãi. Mỗi phiếu đổi trả gắn với phiếu xuất và sản phẩm cụ thể, lưu các thông tin như ngày yêu cầu, lý do, tình trạng sản phẩm, hướng xử lý và trạng thái hoàn tất. Nhờ đó, nhân viên dễ tra cứu giao dịch gốc, kiểm soát hàng hoàn về kho hoặc đổi sang hàng khác, đồng thời hạn chế sai sót khi xử lý thủ công.

3.2.15.1 Thêm Đổi Trả

THÊM PHIẾU ĐỔI TRẢ MỚI

THÊM PHIẾU ĐỔI TRẢ MỚI

Mã phiếu đổi trả:

DT004

Chọn Phiếu Xuất:

...

Mã khách hàng:

Ngày bán:

Hạn cuối đổi trả:

Mã IMEI máy trả:

...

Mã IMEI thay thế:

...

(Tùy chọn - Để trống nếu chưa có máy thay thế)

Ngày đổi trả:

17/03/2026

Lý do:

HỦY

LƯU THAY ĐỔI

Hình 51 Hộp thoại thêm đổi trả cho sản phẩm lỗi

Khi tạo phiếu đổi trả, nhân viên chọn phiếu xuất (hoặc thông tin khách) → chọn sản phẩm trong chi tiết phiếu xuất → nhập số lượng, lý do, tình trạng → hệ thống kiểm tra điều kiện (đã mua chưa, còn trong thời hạn/điều kiện đổi trả, số lượng hợp lệ) → chọn hướng xử lý theo 1 trong 3 case:

Case 1: Đổi hàng (đổi sản phẩm mới/cùng loại):

Hệ thống tạo phiếu đổi trả, trừ tồn kho của hàng đổi ra và nhập lại hàng khách trả (nếu có nhận lại), đồng thời ghi nhận chênh lệch tiền nếu đổi sang sản phẩm khác giá.

Case 2: Trả hàng hoàn tiền:

Hệ thống lập phiếu, cập nhật trạng thái “hoàn tiền”, nhập lại tồn kho nếu hàng trả đủ điều kiện nhập kho (hoặc đánh dấu hư hỏng nếu không), và lưu số tiền hoàn/giảm trừ.

Case 3: Từ chối đổi trả / chuyển bảo hành:

Nếu sản phẩm không đủ điều kiện (quá hạn, lỗi do người dùng, thiếu phụ kiện...) hệ thống ghi nhận phiếu với trạng thái từ chối; trường hợp lỗi kỹ thuật còn bảo hành thì chuyển hướng sang bảo hành để xử lý theo quy trình bảo hành.\

Phụ lục: Code demo chương trình

Link github: https://github.com/PhucTruong11/sound_store_manage.git

1. ChiTietInterface:

```
public interface ChiTietInterface<T> {  
    public int insert(ArrayList<T> t);  
    public int delete(String id);  
    public int update(ArrayList<T> t, String pk);  
    public ArrayList<T> selectAll(String id);  
}
```

2. DAOInterface:

```
public interface DAOInterface<T> {  
    public int insert(T t);  
    public int update(T t);  
    public int delete(String id);  
    public ArrayList<T> selectAll();  
    public T selectById(String id);  
}
```

3. BaoHanhDAO

```
public class BaoHanhDAO implements DAOInterface<BaoHanh> {  
    @Override  
    public int insert(BaoHanh bh) {  
        String sql = "INSERT INTO BaoHanh (MaBH, MaImei, MaPhieuXuat,  
        NgayBatDau, NgayKetThuc) VALUES (?, ?, ?, ?, ?)";  
        try (Connection conn = DatabaseHelper.getConnection();  
            PreparedStatement stmt = conn.prepareStatement(sql)) {  
            stmt.setString(1, bh.getMaBH());  
            stmt.setString(2, bh.getMaImei());  
            stmt.setString(3, bh.getMaPhieuXuat());  
            stmt.setDate(4, Date.valueOf(bh.getNgayBatDau()));  
            stmt.setDate(5, Date.valueOf(bh.getNgayKetThuc()));  
  
            System.out.println("Inserting: " + bh.getMaBH() + ", " +  
            bh.getMaImei());  
            return stmt.executeUpdate();  
        } catch (Exception e) {
```

```

        System.err.println("Lỗi insert BaoHanh:");
        e.printStackTrace();
        return 0;
    }
}

@Override
public ArrayList< BaoHanh > selectAll() {
    ArrayList< BaoHanh > list = new ArrayList<>();
    String sql = "SELECT bh.MaBH, bh.MaImei, bh.MaPhieuXuat,
bh.NgayBatDau, bh.NgayKetThuc, " +
        "COALESCE(ct.TinhTrang, bh.TinhTrang, 'Còn bảo hành') AS
TinhTrangCuoi " +
        "FROM BaoHanh bh " +
        "LEFT JOIN ChiTietBaoHanh ct ON ct.MaCTBH = (" +
        "    SELECT MaCTBH FROM ChiTietBaoHanh " +
        "    WHERE MaBH = bh.MaBH " +
        "    ORDER BY MaCTBH DESC LIMIT 1" +
        ") " +
        "ORDER BY CAST(SUBSTRING(bh.MaBH, 3) AS UNSIGNED) DESC";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            BaoHanh bh = new BaoHanh();
            bh.setMaBH(rs.getString("MaBH"));
            bh.setMaImei(rs.getString("MaImei"));
            bh.setMaPhieuXuat(rs.getString("MaPhieuXuat"));
            bh.setNgayBatDau(rs.getDate("NgayBatDau").toLocalDate());
            bh.setNgayKetThuc(rs.getDate("NgayKetThuc").toLocalDate());
            String tt = rs.getString("TinhTrangCuoi");
            bh.setTinhTrang(tt != null ? tt : "Hoàn thành");
            list.add(bh);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

@Override
public int update(BaoHanh bh) {
    String sql = "UPDATE BaoHanh SET MaImei=?, MaPhieuXuat=?,
NgayBatDau=?, NgayKetThuc=? WHERE MaBH=?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

```

```

        stmt.setString(1, bh.getMaImei());
        stmt.setString(2, bh.getMaPhieuXuat());
        stmt.setDate(3, Date.valueOf(bh.getNgayBatDau()));
        stmt.setDate(4, Date.valueOf(bh.getNgayKetThuc()));
        stmt.setString(5, bh.getMaBH());
        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

@Override
public int delete(String id) {
    String sql = "DELETE FROM BaoHanh WHERE MaBH = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, id);
        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

@Override
public BaoHanh selectById(String id) {
    String sql = "SELECT * FROM BaoHanh WHERE MaBH = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, id);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            BaoHanh bh = new BaoHanh();
            bh.setMaBH(rs.getString("MaBH"));
            bh.setMaImei(rs.getString("MaImei"));
            bh.setMaPhieuXuat(rs.getString("MaPhieuXuat"));
            bh.setNgayBatDau(rs.getDate("NgayBatDau").toLocalDate());
            bh.setNgayKetThuc(rs.getDate("NgayKetThuc").toLocalDate());
            return bh;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

```

```

public ArrayList< BaoHanh > selectAllWithDetails() {
    ArrayList< BaoHanh > list = new ArrayList<>();
    String sql = "SELECT bh.MaBH, bh.MaImei, bh.MaPhieuXuat,
bh.NgayBatDau, bh.NgayKetThuc, sp.TenSP, " +
        "COALESCE(ct.TinhTrang, bh.TinhTrang, 'Còn bảo hành') AS
TinhTrang " +
        "FROM BaoHanh bh " +
        "LEFT JOIN ChiTietBaoHanh ct ON ct.MaCTBH = (" +
        "    SELECT MaCTBH FROM ChiTietBaoHanh " +
        "    WHERE MaBH = bh.MaBH " +
        "    ORDER BY MaCTBH DESC LIMIT 1" +
        ") " +
        "LEFT JOIN ChiTietSP ctsp ON bh.MaImei = ctsp.MaImei " +
        "LEFT JOIN PhienBanSP pb ON ctsp.MaPhienBan = pb.MaPhienBan "
+
        "LEFT JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "ORDER BY CAST(SUBSTRING(bh.MaBH, 3) AS UNSIGNED) ASC";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            BaoHanh bh = new BaoHanh();
            bh.setMaBH(rs.getString("MaBH"));
            bh.setMaImei(rs.getString("MaImei"));
            bh.setMaPhieuXuat(rs.getString("MaPhieuXuat"));
            bh.setNgayBatDau(rs.getDate("NgayBatDau").toLocalDate());
            bh.setNgayKetThuc(rs.getDate("NgayKetThuc").toLocalDate());
            bh.setTenSP(rs.getString("TenSP"));
            String tt = rs.getString("TinhTrang");
            bh.setTinhTrang(tt != null ? tt : "Còn bảo hành");
            list.add(bh);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

public boolean checkImeiExists(String maImei) {
    String sql = "SELECT COUNT(*) FROM ChiTietSP WHERE MaImei = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maImei);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            return rs.getInt(1) > 0;
        }
    }
}

```



```

    }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}

public String generateMaBH() {
    String sql = "SELECT MaBH FROM BaoHanh ORDER BY CAST(SUBSTRING(MaBH,
3) AS UNSIGNED) DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            String lastMa = rs.getString("MaBH");
            int num = Integer.parseInt(lastMa.substring(2));
            return String.format("BH%03d", num + 1);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "BH000";
}

public String getMaBHByImei(String maImei) {
    String sql = "SELECT MaBH FROM BaoHanh WHERE MaImei = ? AND TrangThai
= TRUE ORDER BY MaBH DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maImei);
        ResultSet rs = stmt.executeQuery();
        if (rs.next())
            return rs.getString("MaBH");
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}
}

```

4. ChiTietBaoHanhDAO

```

public class ChiTietBaoHanhDAO implements ChiTietInterface<ChiTietBaoHanh> {
    @Override
    public int insert(ArrayList<ChiTietBaoHanh> list) {
        int count = 0;
        String sql = "INSERT INTO ChiTietBaoHanh VALUES (?, ?, ?, ?)";
    }
}

```

```

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            for (ChiTietBaoHanh ctbh : list) {
                stmt.setString(1, ctbh.getMaCTBH());
                stmt.setString(2, ctbh.getMaBH());
                stmt.setString(3, ctbh.getNoiDung());
                stmt.setString(4, ctbh.getTinhTrang());
                count += stmt.executeUpdate();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return count;
    }

    @Override
    public ArrayList<ChiTietBaoHanh> selectAll(String maBH) {
        ArrayList<ChiTietBaoHanh> list = new ArrayList<>();
        String sql = "SELECT c.*, sp.TenSP " +
            "FROM ChiTietBaoHanh c " +
            "JOIN BaoHanh bh ON c.MaBH = bh.MaBH " +
            "LEFT JOIN ChiTietSP ctsp ON bh.MaImei = ctsp.MaImei " +
            "LEFT JOIN PhienBanSP pb ON ctsp.MaPhienBan = pb.MaPhienBan "
+
            "LEFT JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
            "WHERE c.MaBH = ? " +
            "ORDER BY c.MaCTBH";

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maBH);
            ResultSet rs = stmt.executeQuery();

            while (rs.next()) {
                ChiTietBaoHanh ctbh = new ChiTietBaoHanh();
                ctbh.setMaCTBH(rs.getString("MaCTBH"));
                ctbh.setMaBH(rs.getString("MaBH"));
                ctbh.setTenSP(rs.getString("TenSP"));
                ctbh.setNoiDung(rs.getString("NoiDung"));
                ctbh.setTinhTrang(rs.getString("TinhTrang"));
                list.add(ctbh);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }
}

```

```

    public int update(ChiTietBaoHanh ctbh) {
        String sql = "UPDATE ChiTietBaoHanh SET NoiDung=?, TinhTrang=? WHERE MaCTBH=?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, ctbh.getNoiDung());
            stmt.setString(2, ctbh.getTinhTrang());
            stmt.setString(3, ctbh.getMaCTBH());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    @Override
    public int delete(String id) {
        return 0;
    }

    @Override
    public int update(ArrayList<ChiTietBaoHanh> t, String pk) {
        return 0;
    }

    public String generateMaCTBH() {
        String sql = "SELECT MaCTBH FROM ChiTietBaoHanh ORDER BY CAST(SUBSTRING(MaCTBH, 5) AS UNSIGNED) DESC LIMIT 1";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                String lastMa = rs.getString("MaCTBH");
                int num = Integer.parseInt(lastMa.substring(4));
                return String.format("CTBH%03d", num + 1);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return "CTBH000";
    }
}

```

5. ChiTietPhieuNhapDAO

```

public class ChiTietPhieuNhapDAO implements ChiTietInterface<ChiTietPhieuNhap>
{

```

```

@Override
public int insert(ArrayList<ChiTietPhieuNhap> list) {
    int count = 0;
    String sql = "INSERT INTO ChiTietPhieuNhap VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        for (ChiTietPhieuNhap ct : list) {
            stmt.setString(1, ct.getMaPhieuNhap());
            stmt.setString(2, ct.getMaPhienBan());
            stmt.setInt(3, ct.getSoLuong());
            stmt.setDouble(4, ct.getDonGia());
            stmt.setDouble(5, ct.getThanhTien());
            count += stmt.executeUpdate();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return count;
}

@Override
public ArrayList<ChiTietPhieuNhap> selectAll(String maPN) {
    ArrayList<ChiTietPhieuNhap> list = new ArrayList<>();
    String sql = "SELECT ct.*, sp.TenSP, sp.HinhAnh FROM ChiTietPhieuNhap
ct " +
        "JOIN PhienBanSP pb ON ct.MaPhienBan = pb.MaPhienBan " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "WHERE ct.MaPhieuNhap = ?";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maPN);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            ChiTietPhieuNhap ct = new ChiTietPhieuNhap(
                rs.getString("MaPhieuNhap"),
                rs.getString("MaPhienBan"),
                rs.getInt("SoLuong"),
                rs.getDouble("DonGia"),
                rs.getDouble("ThanhTien"));
            ct.setTenSP(rs.getString("TenSP"));
            ct.setHinhAnh(rs.getString("HinhAnh"));
            list.add(ct);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

        return list;
    }

    @Override
    public int delete(String id) {
        return 0;
    }

    @Override
    public int update(ArrayList<ChiTietPhieuNhap> t, String pk) {
        return 0;
    }
}

```

6. ChiTietPhieuXuatDAO

```

public class ChiTietPhieuXuatDAO implements ChiTietInterface<ChiTietPhieuXuat>
{
    @Override
    public int insert(ArrayList<ChiTietPhieuXuat> list) {
        int count = 0;
        String sql = "INSERT INTO ChiTietPhieuXuat VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            for (ChiTietPhieuXuat ct : list) {
                stmt.setString(1, ct.getMaPhieuXuat());
                stmt.setString(2, ct.getMaPhienBan());
                stmt.setInt(3, ct.getSoLuong());
                stmt.setDouble(4, ct.getDonGia());
                stmt.setDouble(5, ct.getThanhTien());
                count += stmt.executeUpdate();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return count;
    }

    @Override
    public ArrayList<ChiTietPhieuXuat> selectAll(String maPX) {
        ArrayList<ChiTietPhieuXuat> list = new ArrayList<>();
        String sql = "SELECT ct.*, sp.TenSP, sp.HinhAnh FROM ChiTietPhieuXuat
ct " +

```

```

        "JOIN PhienBanSP pb ON ct.MaPhienBan = pb.MaPhienBan " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "WHERE ct.MaPhieuXuat = ?";

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, maPX);

            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    ChiTietPhieuXuat ctpx = new ChiTietPhieuXuat();
                    ctpx.setMaPhieuXuat(rs.getString("MaPhieuXuat"));
                    ctpx.setMaPhienBan(rs.getString("MaPhienBan"));
                    ctpx.setSoLuong(rs.getInt("SoLuong"));
                    ctpx.setDonGia(rs.getDouble("DonGia"));
                    ctpx.setThanhTien(rs.getDouble("ThanhTien"));

                    ctpx.setTenSP(rs.getString("TenSP"));
                    ctpx.setHinhAnh(rs.getString("HinhAnh"));
                    list.add(ctpx);
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public int delete(String id) {
        return 0;
    }

    @Override
    public int update(ArrayList<ChiTietPhieuXuat> t, String pk) {
        return 0;
    }
}

```

7. ChiTietQuyenDAO

```

public class ChiTietQuyenDAO {

    public ArrayList<ChiTietQuyen> selectByMaNQ(String maNQ) {
        ArrayList<ChiTietQuyen> list = new ArrayList<>();
        String sql = "SELECT * FROM ChiTietQuyen WHERE MaNhomQuyen = ?";
    }
}

```

```

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maNQ);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                list.add(new ChiTietQuyén(rs.getString(1),
rs.getString(2), rs.getString(3)));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    public boolean savePermissions(String maNQ, ArrayList<ChiTietQuyén>
dsQuyén) {
        Connection conn = null;
        try {
            conn = DatabaseHelper.getConnection();
            conn.setAutoCommit(false);

            String sqlDelete = "DELETE FROM ChiTietQuyén WHERE
MaNhomQuyén = ?";
            try (PreparedStatement pstDel =
conn.prepareStatement(sqlDelete)) {
                pstDel.setString(1, maNQ);
                pstDel.executeUpdate();
            }

            String sqlInsert = "INSERT INTO ChiTietQuyén (MaNhomQuyén,
MaChucNang, HanhDong) VALUES (?, ?, ?)";
            try (PreparedStatement pstIns =
conn.prepareStatement(sqlInsert)) {
                for (ChiTietQuyén q : dsQuyén) {
                    pstIns.setString(1, q.getMaNhomQuyén());
                    pstIns.setString(2, q.getMaChucNang());
                    pstIns.setString(3, q.getHanhDong());
                    pstIns.addBatch();
                }
                pstIns.executeBatch();
            }

            conn.commit();
            return true;
        } catch (Exception e) {
            if (conn != null) {
                try {
                    conn.rollback();

```

```

        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
    e.printStackTrace();
    return false;
}
}
}

```

8. ChiTietSanPhamDAO

```

public class ChiTietSPDAO implements DAOInterface<ChiTietSP> {

    @Override
    public ArrayList<ChiTietSP> selectAll() {
        return new ArrayList<>();
    }

    @Override
    public ChiTietSP selectById(String id) {
        ChiTietSP ketQua = null;
        String sql = "SELECT * FROM ChiTietSP WHERE MaImei=? WHERE TrangThai=TRUE";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, id);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                ketQua = new ChiTietSP(
                    rs.getString("MaImei"),
                    rs.getString("MaPhienBan"),
                    rs.getString("MaPhieuNhap"),
                    rs.getString("MaPhieuXuat"),
                    rs.getString("TinhTrang"),
                    rs.getBoolean("TrangThai"));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return ketQua;
    }

    public ArrayList<ChiTietSP> selectByMaPB(String maPhienBan) {
        ArrayList<ChiTietSP> list = new ArrayList<>();
        String sql = "SELECT * FROM ChiTietSP WHERE MaPhienBan=? AND TrangThai=TRUE";
    }
}

```



```

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, maPhienBan);
            ResultSet rs = stmt.executeQuery();

            while (rs.next()) {
                ChiTietSP ct = new ChiTietSP(
                    rs.getString("MaImei"),
                    rs.getString("MaPhienBan"),
                    rs.getString("MaPhieuNhap"),
                    rs.getString("MaPhieuXuat"),
                    rs.getString("TinhTrang"),
                    rs.getBoolean("TrangThai"));
                list.add(ct);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public int insert(ChiTietSP ct) {
        String sql = "INSERT INTO ChiTietSP(MaImei, MaPhienBan, MaPhieuNhap, MaPhieuXuat, TinhTrang) VALUES(?,?,?,?,?)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, ct.getMaImei());
            stmt.setString(2, ct.getMaPhienBan());
            stmt.setString(3, ct.getMaPhieuNhap());
            stmt.setString(4, ct.getMaPhieuXuat());
            stmt.setString(5, ct.getTinhTrang());

            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return 0;
    }

    @Override
    public int update(ChiTietSP ct) {
        // Chỉ cho phép cập nhật Tình Trạng dựa theo IMEI
        String sql = "UPDATE ChiTietSP SET TinhTrang=? WHERE MaImei=?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

```

```

        stmt.setString(1, ct.getTinhTrang());
        stmt.setString(2, ct.getMaImei());

        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

@Override
public int delete(String id) {
    String sql = "UPDATE ChiTietSP SET TrangThai=FALSE WHERE MaImei=? ";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, id);
        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public int deleteByMaPB(String maSP) {
    String sql = "UPDATE ChiTietSP SET TrangThai = FALSE WHERE MaPhienBan
= ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maSP);
        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public boolean checkImeiExists(String imei) {
    String sql = "SELECT 1 FROM ChiTietSP WHERE MaImei = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, imei);
        try (ResultSet rs = stmt.executeQuery()) {
            return rs.next();
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

```

```

    }

    public void insertImei(String imei, String maPB, String maPN) {
        String sql = "INSERT INTO ChiTietSP (MaImei, MaPhienBan, MaPhieuNhap,
        TinhTrang, TrangThai) VALUES (?, ?, ?, ?, TRUE)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, imei);
            stmt.setString(2, maPB);
            stmt.setString(3, maPN);
            stmt.setString(4, "Trong kho");
            stmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public ArrayList<String> getImeisByDetails(String maPhieuNhap, String
    maPhienBan) {
        ArrayList<String> list = new ArrayList<>();
        String sql = "SELECT MaImei FROM ChiTietSP WHERE MaPhieuNhap = ? AND
        MaPhienBan = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maPhieuNhap);
            stmt.setString(2, maPhienBan);
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    list.add(rs.getString("MaImei"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }

    public ArrayList<String> getImeisByMaPX(String maPhieuXuat, String
    maPhienBan) {
        ArrayList<String> list = new ArrayList<>();
        String sql = "SELECT MaImei FROM ChiTietSP WHERE MaPhieuXuat = ? AND
        MaPhienBan = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maPhieuXuat);
            stmt.setString(2, maPhienBan);
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {

```

```

        list.add(rs.getString("MaImei"));
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
return list;
}
}

```

9. ChucNangDAO

```

public class ChucNangDAO implements DAOInterface<ChucNang> {

    @Override
    public ArrayList<ChucNang> selectAll() {
        ArrayList<ChucNang> list = new ArrayList<>();
        String sql = "SELECT * FROM ChucNang";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                list.add(new ChucNang(rs.getString(1),
rs.getString(2), rs.getString(3)));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public ChucNang selectById(String id) {
        String sql = "SELECT * FROM ChucNang WHERE MaChucNang = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, id);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                return new ChucNang(rs.getString(1), rs.getString(2),
rs.getString(3));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }
}

```

```

    }

    @Override
    public int insert(ChucNang cn) {
        return 0;
    }

    @Override
    public int update(ChucNang cn) {
        return 0;
    }

    @Override
    public int delete(String id) {
        return 0;
    }
}

```

10. ConNguoiDAO

```

public class ConNguoiDAO {

    public ArrayList<ConNguoi> selectAll() {
        ArrayList<ConNguoi> list = new ArrayList<>();
        String sql = "SELECT * FROM ConNguoi";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement st = conn.prepareStatement(sql);
            ResultSet rs = st.executeQuery()) {
            while (rs.next()) {
                list.add(new ConNguoi(
                    rs.getString("ID"),
                    rs.getString("HoTen"),
                    rs.getString("SDT"),
                    rs.getString("DiaChi")
                ));
            }
        } catch (Exception e) { e.printStackTrace(); }
        return list;
    }

    public int insert(ConNguoi cn) {
        String sql = "INSERT INTO ConNguoi(ID, HoTen, SDT, DiaChi) VALUES(?,?,?,?)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, cn.getId());
            ps.setString(2, cn.getHoTen());

```

```

        ps.setString(3, cn.getSdt());
        ps.setString(4, cn.getDiaChi());
        return ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); return 0; }
}

public int update(ConNguoi cn) {
    String sql = "UPDATE ConNguoi SET HoTen=?, SDT=?, DiaChi=? WHERE
ID=?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, cn.getHoTen());
        ps.setString(2, cn.getSdt());
        ps.setString(3, cn.getDiaChi());
        ps.setString(4, cn.getId());
        return ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); return 0;}
}
}

```

11. KhachHangDAO

```

public class KhachHangDAO implements DAOInterface<KhachHang> {

    public ArrayList<KhachHang> selectAll() {
        ArrayList<KhachHang> list = new ArrayList<>();
        String sql = "SELECT c.ID, c.HoTen, c.SDT, c.DiaChi, k.TrangThai " +
            "FROM ConNguoi c JOIN KhachHang k ON c.ID = k.ID " +
            "WHERE k.TrangThai = TRUE";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement st = conn.prepareStatement(sql);
            ResultSet rs = st.executeQuery()) {
            while (rs.next()) {
                list.add(new KhachHang(
                    rs.getString("ID"),
                    rs.getString("HoTen"),
                    rs.getString("SDT"),
                    rs.getString("DiaChi"),
                    rs.getBoolean("TrangThai")
                ));
            }
        } catch (Exception e) { e.printStackTrace(); }
        return list;
    }
}

```

```

public int insert(KhachHang kh) {
    // Insert vào ConNguoi trước
    new ConNguoiDAO().insert(kh);
    String sql = "INSERT INTO KhachHang(ID, TrangThai) VALUES(?,TRUE)";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, kh.getId());
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

public int update(KhachHang kh) {
    // Update bảng ConNguoi trước
    new ConNguoiDAO().update(kh);
    String sql = "UPDATE KhachHang SET TrangThai = ? WHERE ID=?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setBoolean(1, kh.isTrangThai());
        ps.setString(2, kh.getId());
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

public int delete(String id) {
    String sql = "UPDATE KhachHang SET TrangThai=FALSE WHERE ID=?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, id);
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

public String generateMaKH() {
    String sql = "SELECT ID FROM KhachHang ORDER BY CAST(SUBSTRING(ID, 4)
AS UNSIGNED) DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if(rs.next()) {

```

```

        String lastMa = rs.getString("ID");
        int num = Integer.parseInt(lastMa.substring(3));
        return String.format("KH%03d", num + 1);
    }
} catch (Exception e) {
    e.printStackTrace();
}
return "KH00";
}

@Override
public KhachHang selectById(String id) {
    String sql = "SELECT c.ID, c.HoTen, c.SDT, c.DiaChi, k.TrangThai " +
        "FROM ConNguoi c JOIN KhachHang k ON c.ID = k.ID WHERE c.ID = " +
        id + ";";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, id);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            return new KhachHang(
                rs.getString("ID"),
                rs.getString("HoTen"),
                rs.getString("SDT"),
                rs.getString("DiaChi"),
                rs.getBoolean("TrangThai"));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}
}

```

12. DoiTraDAO

```
public class DoiTraDAO {
```



```

public ArrayList<DoiTra> selectAll() {
    ArrayList<DoiTra> list = new ArrayList<>();
    String sql = ""
        SELECT dt.*, px.NgayXuat, cn.HoTen
        FROM DoiTra dt
        JOIN PhieuXuat px ON dt.MaPhieuXuat = px.MaPhieuXuat
        JOIN KhachHang kh ON dt.MaKH = kh.ID
        JOIN ConNguoi cn ON kh.ID = cn.ID
        WHERE dt.TrangThai = 1
    "";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            DoiTra dt = new DoiTra(
                rs.getString("MaDoiTra"),
                rs.getString("MaKH"),
                rs.getString("MaPhieuXuat"),
                rs.getString("MaImei"),
                rs.getDate("NgayDoiTra").toLocalDate(),
                rs.getString("LyDo"),
                rs.getBoolean("TrangThai")
            );
            dt.setTenKH(rs.getString("HoTen"));
            LocalDate ngayMua = rs.getDate("NgayXuat").toLocalDate();
            dt.setNgayMua(ngayMua);
            // ngayHetHan sẽ auto calculate trong setNgayMua (ngayMua +
30)

            list.add(dt);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

public int insert(DoiTra dt) {
    String sql = "INSERT INTO DoiTra (MaDoiTra, MaKH, MaPhieuXuat, MaImei,
NgayDoiTra, LyDo, TrangThai) VALUES (?, ?, ?, ?, ?, ?, 1)";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, dt.getMaDoiTra());
        ps.setString(2, dt.getMaKH());
        ps.setString(3, dt.getMaPhieuXuat());

```

```

        ps.setString(4, dt.getMaImei());
        ps.setDate(5, Date.valueOf(dt.getNgayDoiTra()));
        ps.setString(6, dt.getLyDo());

        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public int update(DoiTra dt) {

    String sql = "UPDATE DoiTra SET NgayDoiTra = ?, LyDo = ? WHERE MaDoiTra = ? AND TrangThai = 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setDate(1, Date.valueOf(dt.getNgayDoiTra()));
        ps.setString(2, dt.getLyDo());
        ps.setString(3, dt.getMaDoiTra());
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public int softDelete(String maDT) {
    String sql = "UPDATE DoiTra SET TrangThai = 0 WHERE MaDoiTra = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, maDT);
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public String getImeiByMaDoiTra(String maDT) {
    String sql = "SELECT MaImei FROM DoiTra WHERE MaDoiTra = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, maDT);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {

```

```

        return rs.getString("MaImei");
    }
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

public boolean restoreImeiStatus(String imei) {
    // Restore máy trả về trạng thái 'Đã bán'
    String sql = "UPDATE ChiTietSP SET TinhTrang = 'Đã bán' WHERE MaImei = ? AND TinhTrang = 'Máy lỗi đổi trả'";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, imei);
        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}

public LocalDate getNgayMuaByMaPX(String maPX) {
    String sql = "SELECT NgayXuat FROM PhieuXuat WHERE MaPhieuXuat = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, maPX);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return rs.getDate("NgayXuat").toLocalDate();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

public boolean updateOldImeiStatus(String imeiCu, String tinhTrangMoi) {
    String sql = "UPDATE ChiTietSP SET TinhTrang = ?, MaPhieuXuat = NULL WHERE MaImei = ?";
    try (java.sql.Connection conn = Backend.DatabaseHelper.getConnection();
        java.sql.PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, tinhTrangMoi);
        ps.setString(2, imeiCu);
    }
}

```

```

        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}

public boolean assignNewImeiToInvoice(String imeiMoi, String maPX) {

    String sql = "UPDATE ChiTietSP SET TinhTrang = 'Đã bán', MaPhieuXuat = ? WHERE MaImei = ?";
    try (java.sql.Connection conn = Backend.DatabaseHelper.getConnection();
        java.sql.PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, maPX);
        ps.setString(2, imeiMoi);

        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}

public boolean checkImeiInvoice(String imei, String maPX) {
    String sql = "SELECT COUNT(*) FROM ChiTietSP WHERE MaImei = ? AND MaPhieuXuat = ?";
    try (java.sql.Connection conn = Backend.DatabaseHelper.getConnection();
        java.sql.PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, imei);
        ps.setString(2, maPX);

        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return rs.getInt(1) > 0;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}

// Check if IMEI exists in kho (Trong kho)

```

```

    public boolean checkImeiInStock(String imei) {
        String sql = "SELECT COUNT(*) FROM ChiTietSP WHERE MaImei = ? AND  

TinhTrang = 'Trong kho'";
        try (java.sql.Connection conn =  

Backend.DatabaseHelper.getConnection();
            java.sql.PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, imei);

            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return rs.getInt(1) > 0;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return false;
    }

    public java.util.HashMap<String, String> getProductInfoByImei(String imei)
{
        java.util.HashMap<String, String> result = new java.util.HashMap<>();
        String sql = ""
            SELECT
                sp.TenSP,
                sp.HinhAnh,
                pb.MauSac,
                pb.CongSuat,
                pb.GiaBan,
                ct.TinhTrang
            FROM ChiTietSP ct
            JOIN PhienBanSP pb ON ct.MaPhienBan = pb.MaPhienBan
            JOIN SanPham sp ON pb.MaSP = sp.MaSP
            WHERE ct.MaImei = ?
            """;
        try (java.sql.Connection conn =  

Backend.DatabaseHelper.getConnection();
            java.sql.PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, imei);

            java.sql.ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                result.put("tenSP", rs.getString("TenSP") != null ?  

rs.getString("TenSP") : "N/A");
                result.put("hinhAnh", rs.getString("HinhAnh") != null ?  

rs.getString("HinhAnh") : "");
            }
        }
    }

```

```

        result.put("mauSac", rs.getString("MauSac") != null ?
rs.getString("MauSac") : "N/A");
        result.put("congSuat", rs.getString("CongSuat") != null ?
rs.getString("CongSuat") : "N/A");
        result.put("giaBan", String.valueOf(rs.getDouble("GiaBan")));
        result.put("tinhTrang", rs.getString("TinhTrang") != null ?
rs.getString("TinhTrang") : "N/A");
    }
} catch (Exception e) {
    e.printStackTrace();
}
return result;
}

public String getMaDoiTraByImei(String imei) {
    String sql = "SELECT MaDoiTra FROM DoiTra WHERE MaImei = ? AND
TrangThai = 1 LIMIT 1";
    try (java.sql.Connection conn =
Backend.DatabaseHelper.getConnection();
        java.sql.PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, imei);

        java.sql.ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return rs.getString("MaDoiTra");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

public String getMaPhienBanByImei(String imei) {
    String sql = "SELECT MaPhienBan FROM ChiTietSP WHERE MaImei = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, imei);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) return rs.getString("MaPhienBan");
    } catch (Exception e) { e.printStackTrace(); }
    return null;
}

public void updateQuantityInChiTietPX(String maPX, String maPB, int
thayDoi) {

```

```

try (Connection conn = DatabaseHelper.getConnection()) {

    String checkSql = "SELECT SoLuong, DonGia FROM ChiTietPhieuXuat
WHERE MaPhieuXuat = ? AND MaPhienBan = ?";
    PreparedStatement checkPs = conn.prepareStatement(checkSql);
    checkPs.setString(1, maPX);
    checkPs.setString(2, maPB);
    ResultSet rs = checkPs.executeQuery();

    if (rs.next()) {

        int currentQty = rs.getInt("SoLuong");
        double unitPrice = rs.getDouble("DonGia");
        int newQty = rs.getInt("SoLuong") + thayDoi;

        if (newQty > 0) {
            String updateSql = "UPDATE ChiTietPhieuXuat SET SoLuong =
?, ThanhTien = ? WHERE MaPhieuXuat = ? AND MaPhienBan = ?";
            PreparedStatement updatePs =
conn.prepareStatement(updateSql);
            updatePs.setInt(1, newQty);
            updatePs.setDouble(2, newQty * unitPrice);
            updatePs.setString(3, maPX);
            updatePs.setString(4, maPB);
            updatePs.executeUpdate();
        } else {

            String deleteSql = "DELETE FROM ChiTietPhieuXuat WHERE
MaPhieuXuat = ? AND MaPhienBan = ?";
            PreparedStatement deletePs =
conn.prepareStatement(deleteSql);
            deletePs.setString(1, maPX);
            deletePs.setString(2, maPB);
            deletePs.executeUpdate();
        }
    } else if (thayDoi > 0) {
        String getPriceSql = "SELECT GiaBan FROM PhienBanSP WHERE
MaPhienBan = ?";
        PreparedStatement getPricePs =
conn.prepareStatement(getPriceSql);
        getPricePs.setString(1, maPB);
        ResultSet rsPrice = getPricePs.executeQuery();

        if (rsPrice.next()) {
            double giaBan = rsPrice.getDouble("GiaBan");
            String insertSql = "INSERT INTO ChiTietPhieuXuat
(MaPhieuXuat, MaPhienBan, SoLuong, DonGia, ThanhTien) VALUES (?, ?, ?, ?, ?)";

```

```

        PreparedStatement insertPs =
conn.prepareStatement(insertSql);
        insertPs.setString(1, maPX);
        insertPs.setString(2, maPB);
        insertPs.setInt(3, thayDoi);
        insertPs.setDouble(4, giaBan);
        insertPs.setDouble(5, thayDoi * giaBan);
        insertPs.executeUpdate();
    }
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

13. KhuyenMaiDAO

```

public class KhuyenMaiDAO {
    public ArrayList<KhuyenMai> getAll() {
        ArrayList<KhuyenMai> list = new ArrayList<>();
        String sql = "SELECT MaKM, TenKM, PhanTramGiam, NgayBatDau,
NgayKetThuc, TrangThai FROM KhuyenMai";
        try (Connection con = DatabaseHelper.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                list.add(new KhuyenMai(
                    rs.getString("MaKM"),
                    rs.getString("TenKM"),
                    rs.getDouble("PhanTramGiam"),
                    rs.getDate("NgayBatDau"),
                    rs.getDate("NgayKetThuc"),
                    rs.getInt("TrangThai")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    public boolean add(KhuyenMai km) {
        String sql = "INSERT INTO KhuyenMai (MaKM, TenKM, PhanTramGiam,
NgayBatDau, NgayKetThuc, TrangThai) VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection con = DatabaseHelper.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, km.getMaKM());

```



```

        ps.setString(2, km.getTenKM());
        ps.setDouble(3, km.getPhanTramGiam());
        ps.setDate(4, new java.sql.Date(km.getNgayBD().getTime()));
        ps.setDate(5, new java.sql.Date(km.getNgayKT().getTime()));
        ps.setInt(6, 1);
        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        return false;
    }
}

public boolean update(KhuyenMai km) {
    String sql = "UPDATE KhuyenMai SET TenKM=?, PhanTramGiam=?,
    NgayBatDau=?, NgayKetThuc=?, TrangThai=? WHERE MaKM=?";
    try (Connection con = DatabaseHelper.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, km.getTenKM());
        ps.setDouble(2, km.getPhanTramGiam());
        ps.setDate(3, new java.sql.Date(km.getNgayBD().getTime()));
        ps.setDate(4, new java.sql.Date(km.getNgayKT().getTime()));
        ps.setInt(5, km.getTrangThai());
        ps.setString(6, km.getMaKM());
        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        return false;
    }
}

public boolean updateStatus(String maKM, int status) {
    String sql = "UPDATE KhuyenMai SET TrangThai = ? WHERE MaKM = ?";
    try (Connection con = DatabaseHelper.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, status);
        ps.setString(2, maKM);
        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        return false;
    }
}

public int autoUpdateExpiredStatus() {
    String sql = "UPDATE KhuyenMai SET TrangThai = 0 WHERE NgayKetThuc <
    CURRENT_DATE AND TrangThai = 1";
    try (Connection con = DatabaseHelper.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

        return 0;
    }
}
}

```

14. LoaiSPDAO

```

1public class LoaiSPDAO {
    public ArrayList<LoaiSP> selectAll() {
        ArrayList<LoaiSP> list = new ArrayList<>();
        String sql = "SELECT * FROM LoaiSP";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                list.add(new LoaiSP(rs.getString("MaLoai"),
rs.getString("TenLoai")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }
}

```

15. NhaCungCapDAO

```

public class NhaCungCapDAO implements DAOInterface<NhaCungCap> {
    @Override
    public int insert(NhaCungCap ncc) {
        String sql = "INSERT INTO NhaCungCap (MaNCC, TenNCC, DiaChi, Sdt,
TrangThai) VALUES (?, ?, ?, ?, TRUE)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, ncc.getMaNCC());
            stmt.setString(2, ncc.getTenNCC());
            stmt.setString(3, ncc.getDiaChi());
            stmt.setString(4, ncc.getSdt());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }
}

```

```

    }

    @Override
    public ArrayList<NhaCungCap> selectAll() {
        ArrayList<NhaCungCap> list = new ArrayList<>();
        String sql = "SELECT * FROM NhaCungCap WHERE TrangThai = TRUE ORDER BY
TenNCC DESC";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                list.add(new NhaCungCap(
                    rs.getString(1),
                    rs.getString(2),
                    rs.getString(3),
                    rs.getString(4),
                    rs.getBoolean(5)));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public int update(NhaCungCap ncc) {
        String sql = "UPDATE NhaCungCap SET TenNCC=?, DiaChi=?, SDT=? WHERE
MaNCC=?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, ncc.getTenNCC());
            stmt.setString(2, ncc.getDiaChi());
            stmt.setString(3, ncc.getSdt());
            stmt.setString(4, ncc.getMaNCC());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    @Override
    public int delete(String id) {
        String sql = "UPDATE NhaCungCap SET TrangThai = FALSE WHERE MaNCC =
?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, id);

```

```

        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

@Override
public NhaCungCap selectById(String id) {
    String sql = "SELECT * FROM NhaCungCap WHERE MaNCC = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, id);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            return new NhaCungCap(
                rs.getString("MaNCC"),
                rs.getString("TenNCC"),
                rs.getString("DiaChi"),
                rs.getString("Sdt"),
                rs.getBoolean("TrangThai"));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

public String generateMaNCC() {
    String sql = "SELECT MaNCC FROM NhaCungCap ORDER BY CAST(SUBSTRING(MaNCC, 4) AS UNSIGNED) DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            String lastMa = rs.getString("MaNCC");
            int num = Integer.parseInt(lastMa.substring(3));
            return String.format("NCC%03d", num + 1);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "NCC000";
}

public ArrayList<String> getMaSPByNCC(String maNCC) {
    ArrayList<String> list = new ArrayList<>();

```

```

String sql = "SELECT MaSP FROM NCC_SanPham WHERE MaNCC = ?";
try (Connection conn = DatabaseHelper.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, maNCC);
    ResultSet rs = stmt.executeQuery();
    while (rs.next()) list.add(rs.getString("MaSP"));
} catch (Exception e) {
    e.printStackTrace();
}
return list;
}

public boolean updateSanPhamCungCap(String maNCC, ArrayList<String>
dsMaSP) {
String sqlDelete = "DELETE FROM NCC_SanPham WHERE MaNCC = ?";
String sqlInsert = "INSERT INTO NCC_SanPham (MaNCC, MaSP) VALUES (?,
?)";

try (Connection conn = DatabaseHelper.getConnection()) {
    conn.setAutoCommit(false);

    try (PreparedStatement psDel = conn.prepareStatement(sqlDelete)) {
        psDel.setString(1, maNCC);
        psDel.executeUpdate();
    }

    try (PreparedStatement psIns = conn.prepareStatement(sqlInsert)) {
        for (String maSP : dsMaSP) {
            psIns.setString(1, maNCC);
            psIns.setString(2, maSP);
            psIns.addBatch();
        }
        psIns.executeBatch();
    }

    conn.commit();
    return true;
} catch (Exception e) { e.printStackTrace(); return false; }
}
}

```

16. NhanVienDAO

```

public class NhanVienDAO implements DAOInterface<NhanVien> {

    public ArrayList<NhanVien> selectAll() {

```

```

        ArrayList<NhanVien> list = new ArrayList<>();
        String sql = "SELECT c.ID, c.HoTen, c.SDT, c.DiaChi, n.ChucVu,
n.Email, n.Luong, n.TrangThai " +
            "FROM ConNguoi c JOIN NhanVien n ON c.ID = n.ID " +
            "WHERE n.TrangThai = TRUE";

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement st = conn.prepareStatement(sql);
            ResultSet rs = st.executeQuery()) {
            while (rs.next()) {
                list.add(new NhanVien(
                    rs.getString("ID"),
                    rs.getString("HoTen"),
                    rs.getString("SDT"),
                    rs.getString("DiaChi"),
                    rs.getString("ChucVu"),
                    rs.getString("Email"),
                    rs.getDouble("Luong"),
                    rs.getBoolean("TrangThai")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    public int insert(NhanVien nv) {
        // Insert vào ConNguoi trước
        new ConNguoiDAO().insert(nv);
        String sql = "INSERT INTO NhanVien(ID, ChucVu, Email, Luong,
TrangThai) VALUES(?,?,?,?,TRUE)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, nv.getId());
            ps.setString(2, nv.getChucVu());
            ps.setString(3, nv.getEmail());
            ps.setDouble(4, nv.getLuong());
            return ps.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    public int update(NhanVien nv) {
        new ConNguoiDAO().update(nv);
        String sql = "UPDATE NhanVien SET ChucVu=?, Email=?, Luong=? WHERE
ID=?";
    }

```

```

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, nv.getChucVu());
            ps.setString(2, nv.getEmail());
            ps.setDouble(3, nv.getLuong());
            ps.setString(4, nv.getId());
            return ps.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    public int delete(String id) {
        String sql = "UPDATE NhanVien SET TrangThai=FALSE WHERE ID=?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, id);
            return ps.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    public String generateMaNV() {
        String sql = "SELECT ID FROM NhanVien ORDER BY CAST(SUBSTRING(ID, 4)
AS UNSIGNED) DESC LIMIT 1";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                String lastMa = rs.getString("ID");
                int num = Integer.parseInt(lastMa.substring(3));
                return String.format("NV%03d", num + 1);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return "NV00";
    }

    @Override
    public NhanVien selectById(String id) {
        String sql = "SELECT c.ID, c.HoTen, c.SDT, c.DiaChi, n.ChucVu,
n.Email, n.Luong, n.TrangThai " +
            "FROM ConNguoi c JOIN NhanVien n ON c.ID = n.ID WHERE c.ID =
?";
    }

```

```

try (Connection conn = DatabaseHelper.getConnection();
    PreparedStatement st = conn.prepareStatement(sql)) {
    st.setString(1, id);
    ResultSet rs = st.executeQuery();
    if (rs.next()) {
        return new NhanVien(
            rs.getString("ID"),
            rs.getString("HoTen"),
            rs.getString("SDT"),
            rs.getString("DiaChi"),
            rs.getString("ChucVu"),
            rs.getString("Email"),
            rs.getDouble("Luong"),
            rs.getBoolean("TrangThai"));
    }
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

public boolean insertWithAccount(NhanVien nv, TaiKhoan tk) {
    Connection conn = null;
    try {
        conn = DatabaseHelper.getConnection();
        conn.setAutoCommit(false);

        String sqlCN = "INSERT INTO ConNguoi (ID, HoTen, SDT, DiaChi)
VALUES (?, ?, ?, ?)";
        PreparedStatement psCN = conn.prepareStatement(sqlCN);
        psCN.setString(1, nv.getId());
        psCN.setString(2, nv.getHoTen());
        psCN.setString(3, nv.getSdt());
        psCN.setString(4, nv.getDiaChi());
        psCN.executeUpdate();

        String sqlNV = "INSERT INTO NhanVien (ID, ChucVu, Email, Luong,
TrangThai) VALUES (?, ?, ?, ?, TRUE)";
        PreparedStatement psNV = conn.prepareStatement(sqlNV);
        psNV.setString(1, nv.getId());
        psNV.setString(2, nv.getChucVu());
        psNV.setString(3, nv.getEmail());
        psNV.setDouble(4, nv.getLuong());
        psNV.executeUpdate();

        String sqlTK = "INSERT INTO TaiKhoan (TenDangNhap, MatKhai, MaNV,
MaNhomQuyen, TrangThai) VALUES (?, ?, ?, ?, 1)";
        PreparedStatement psTK = conn.prepareStatement(sqlTK);

```



```

        psTK.setString(1, tk.getUsername());
        psTK.setString(2, tk.getPassword());
        psTK.setString(3, tk.getMaNV());
        psTK.setString(4, tk.getMaNhomQuyen());
        psTK.executeUpdate();

        conn.commit();
        return true;
    } catch (Exception e) {
        if (conn != null)
            try {
                conn.rollback();
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        e.printStackTrace();
        return false;
    }
}
}

```

17. NhomQuyenDAO

```

public class NhomQuyenDAO implements DAOInterface<NhomQuyen> {

    @Override
    public int insert(NhomQuyen nq) {
        String sql = "INSERT INTO NhomQuyen (MaNhomQuyen, TenNhomQuyen, MoTa) VALUES (?, ?, ?)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, nq.getMaNhomQuyen());
            stmt.setString(2, nq.getTenNhomQuyen());
            stmt.setString(3, nq.getMoTa());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    @Override
    public ArrayList<NhomQuyen> selectAll() {
        ArrayList<NhomQuyen> list = new ArrayList<>();
        String sql = "SELECT * FROM NhomQuyen";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);

```

```

        ResultSet rs = stmt.executeQuery() {
            while (rs.next()) {
                list.add(new NhomQuyen(
                    rs.getString("MaNhomQuyen"),
                    rs.getString("TenNhomQuyen"),
                    rs.getString("MoTa")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public NhomQuyen selectById(String id) {
        String sql = "SELECT * FROM NhomQuyen WHERE MaNhomQuyen = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, id);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                return new NhomQuyen(rs.getString(1), rs.getString(2),
rs.getString(3));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    @Override
    public int update(NhomQuyen nq) {
        String sql = "UPDATE NhomQuyen SET TenNhomQuyen = ?, MoTa = ?
WHERE MaNhomQuyen = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, nq.getTenNhomQuyen());
            stmt.setString(2, nq.getMoTa());
            stmt.setString(3, nq.getMaNhomQuyen());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    @Override
    public int delete(String id) {

```

```

        String sql = "DELETE FROM NhomQuyen WHERE MaNhomQuyen = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, id);
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    public String generateMaNQ() {
        String sql = "SELECT MaNhomQuyen FROM NhomQuyen ORDER BY
CAST(SUBSTRING(MaNhomQuyen, 3) AS UNSIGNED) DESC LIMIT 1";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                String lastMa = rs.getString(1);
                int num = Integer.parseInt(lastMa.substring(2));
                return String.format("NQ%02d", num + 1);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return "NQ01";
    }

    public ArrayList<ChiTietQuyen> selectQuyenByNhom(String maNQ) {
        ArrayList<ChiTietQuyen> list = new ArrayList<>();
        String sql = "SELECT * FROM ChiTietQuyen WHERE MaNhomQuyen = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maNQ);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                list.add(new ChiTietQuyen(
                    rs.getString("MaNhomQuyen"),
                    rs.getString("MaChucNang"),
                    rs.getString("HanhDong")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }
}

```

18. PhieuNhapDAO

```
public class PhieuNhapDAO implements DAOInterface<PhieuNhap> {

    @Override
    public int insert(PhieuNhap pn) {
        String sql = "INSERT INTO PhieuNhap (MaPhieuNhap, MaNV, MaNCC, TongTien, TrangThai) VALUES (?, ?, ?, ?, TRUE)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, pn.getmaPhieuNhap());
            stmt.setString(2, pn.getmaNV());
            stmt.setString(3, pn.getmaNCC());
            stmt.setDouble(4, pn.getTongTien());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    @Override
    public ArrayList<PhieuNhap> selectAll() {
        ArrayList<PhieuNhap> list = new ArrayList<>();
        String sql = "SELECT * FROM PhieuNhap WHERE TrangThai = TRUE ORDER BY NgayNhap DESC";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                list.add(new PhieuNhap(
                    rs.getString(1),
                    rs.getTimestamp(2),
                    rs.getString(3),
                    rs.getString(4),
                    rs.getDouble(5),
                    rs.getBoolean(6)));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public PhieuNhap selectById(String id) {
        String sql = "SELECT * FROM PhieuNhap WHERE MaPhieuNhap = ?";
        try (Connection conn = DatabaseHelper.getConnection();
```

```

        PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, id);
    ResultSet rs = stmt.executeQuery();

    if (rs.next()) {
        return new PhieuNhap(
            rs.getString("MaPhieuNhap"),
            rs.getTimestamp("NgayNhap"),
            rs.getString("MaNV"),
            rs.getString("MaNCC"),
            rs.getDouble("TongTien"),
            rs.getBoolean("TrangThai"));
    }
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

@Override
public int update(PhieuNhap pn) {
    return 0;
}

@Override
public int delete(String id) {
    throw new UnsupportedOperationException("Không được xóa phiếu!");
}

public String generateMaPhieuNhap() {
    String sql = "SELECT MaPhieuNhap FROM PhieuNhap ORDER BY
CAST(SUBSTRING(MaPhieuNhap, 3) AS UNSIGNED) DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            String lastMa = rs.getString("MaPhieuNhap");
            int num = Integer.parseInt(lastMa.substring(2));
            return String.format("PN%03d", num + 1);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "PN00";
}

public ArrayList<PhieuNhap> selectByNCC(String maNCC) {
    ArrayList<PhieuNhap> list = new ArrayList<>();

```

```

        String sql = "SELECT * FROM PhieuNhap WHERE MaNCC = ? AND TrangThai = TRUE ORDER BY NgayNhap DESC";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maNCC);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                list.add(new PhieuNhap(
                    rs.getString("MaPhieuNhap"),
                    rs.getTimestamp("NgayNhap"),
                    rs.getString("MaNV"),
                    rs.getString("MaNCC"),
                    rs.getDouble("TongTien"),
                    rs.getBoolean("TrangThai")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    public ArrayList<PhieuNhap> selectByFilter(/* String maNCC, */ Date from,
        Date to, Long minPrice, Long maxPrice) {
        ArrayList<PhieuNhap> list = new ArrayList<>();
        StringBuilder sql = new StringBuilder("SELECT * FROM PhieuNhap WHERE TrangThai = TRUE");

        // if(!maNCC.equals("All")) sql.append(" AND MaNCC = ?");
        if (from != null)
            sql.append(" AND NgayNhap >= ?");
        if (to != null)
            sql.append(" AND NgayNhap <= ?");
        if (minPrice >= 0)
            sql.append(" AND TongTien >= ?");
        if (maxPrice > 0)
            sql.append(" AND TongTien <= ?");

        sql.append(" ORDER BY NgayNhap DESC");

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt =
conn.prepareStatement(sql.toString())) {
            int index = 1;
            // if(!maNCC.equals("All")) stmt.setString(index++, maNCC);
            if (from != null)
                stmt.setTimestamp(index++, new Timestamp(from.getTime()));
            if (to != null)
                stmt.setTimestamp(index++, new Timestamp(to.getTime()));

```

```

        if (minPrice >= 0)
            stmt.setLong(index++, minPrice);
        if (maxPrice > 0)
            stmt.setLong(index++, maxPrice);

        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            list.add(new PhieuNhap(rs.getString(1), rs.getTimestamp(2),
rs.getString(3),
                                rs.getString(4), rs.getDouble(5), rs.getBoolean(6)));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}
}

```

19. PhieuXuatDAO

```

public class PhieuXuatDAO implements DAOInterface<PhieuXuat> {
    @Override
    public ArrayList<PhieuXuat> selectAll() {
        ArrayList<PhieuXuat> list = new ArrayList<>();
        String sql = "SELECT * FROM PhieuXuat";

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                PhieuXuat px = new PhieuXuat();

                px.setMaPhieuXuat(rs.getString("MaPhieuXuat"));
                px.setNgayXuat(rs.getTimestamp("NgayXuat"));
                px.setMaNV(rs.getString("MaNV"));
                px.setMaKH(rs.getString("MaKH"));
                px.setMaKM(rs.getString("MaKM"));
                px.setTongTien(rs.getDouble("TongTien"));
                px.setTrangThai(rs.getBoolean("TrangThai"));

                list.add(px);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```

        return list;
    }

    @Override
    public int insert(PhieuXuat px) {
        String sql = "INSERT INTO PhieuXuat (MaPhieuXuat, NgayXuat, MaNV, MaKH, MaKM, TongTien, TrangThai) VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, px.getMaPhieuXuat());
            stmt.setTimestamp(2, px.getNgayXuat());
            stmt.setString(3, px.getMaNV());
            stmt.setString(4, px.getMaKH());
            stmt.setString(5, px.getMaKM());
            stmt.setDouble(6, px.getTongTien());
            stmt.setBoolean(7, px.getTrangThai());
            return stmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
            return 0;
        }
    }

    @Override
    public PhieuXuat selectById(String id) {
        String sql = "SELECT * FROM PhieuXuat WHERE MaPhieuXuat = ?";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, id);
            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                return new PhieuXuat(
                    rs.getString("MaPhieuXuat"),
                    rs.getTimestamp("NgayXuat"),
                    rs.getString("MaNV"),
                    rs.getString("MaKH"),
                    rs.getString("MaKM"),
                    rs.getDouble("TongTien"),
                    rs.getBoolean("TrangThai"));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    @Override

```



```

public int update(PhieuXuat px) {
    return 0;
}

@Override
public int delete(String id) {
    throw new UnsupportedOperationException("Không được xóa phiếu!");
}

public String generateMaPhieuXuat() {
    String sql = "SELECT MaPhieuXuat FROM PhieuXuat ORDER BY
CAST(SUBSTRING(MaPhieuXuat, 3) AS UNSIGNED) DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            String lastMa = rs.getString("MaPhieuXuat");
            int num = Integer.parseInt(lastMa.substring(2));
            return String.format("PX%03d", num + 1);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "PX000";
}

public ArrayList<PhieuXuat> search(String keyword) {
    ArrayList<PhieuXuat> list = new ArrayList<>();
    String sql = "SELECT * FROM PhieuXuat WHERE MaPhieuXuat LIKE ? OR MaKH
LIKE ?";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, "%" + keyword + "%");
        stmt.setString(2, "%" + keyword + "%");

        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                PhieuXuat px = new PhieuXuat();
                px.setMaPhieuXuat(rs.getString("MaPhieuXuat"));
                px.setNgayXuat(rs.getTimestamp("NgayXuat"));
                px.setMaNV(rs.getString("MaNV"));
                px.setMaKH(rs.getString("MaKH"));
                px.setMaKM(rs.getString("MaKM"));
                px.setTongTien(rs.getDouble("TongTien"));
                list.add(px);
            }
        }
    }
}

```

```

    }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

public boolean thanhToan(PhieuXuat px, ArrayList<ChiTietPhieuXuat>
dsChiTiet) {
    Connection conn = null;
    try {
        conn = DatabaseHelper.getConnection();
        conn.setAutoCommit(false);

        String sqlPX = "INSERT INTO PhieuXuat (MaPhieuXuat, NgayXuat,
MaNV, MaKH, MaKM, TongTien, TrangThai) VALUES (?, ?, ?, ?, ?, ?, ?)";
        PreparedStatement pstPX = conn.prepareStatement(sqlPX);
        pstPX.setString(1, px.getMaPhieuXuat());
        pstPX.setTimestamp(2, px.getNgayXuat());
        pstPX.setString(3, px.getMaNV());
        pstPX.setString(4, px.getMaKH());
        pstPX.setString(5, px.getMaKM());
        pstPX.setDouble(6, px.getTongTien());
        pstPX.setBoolean(7, true);
        pstPX.executeUpdate();

        String sqlCT = "INSERT INTO ChiTietPhieuXuat (MaPhieuXuat,
MaPhienBan, SoLuong, DonGia) VALUES (?, ?, ?, ?)";
        String sqlSelectImei = "SELECT MaImei FROM ChiTietSP WHERE
MaPhienBan = ? AND TinhTrang = 'Trong kho' LIMIT ?";
        String sqlUpdateImei = "UPDATE ChiTietSP SET TinhTrang = 'Đã bán',
MaPhieuXuat = ? WHERE MaImei = ?";
        String sqlInsertBH = "INSERT INTO BaoHanh (MaBH, MaImei,
MaPhieuXuat, NgayBatDau, NgayKetThuc) VALUES (?, ?, ?, ?, ?)";

        String sqlInsertCTBH = "INSERT INTO ChiTietBaoHanh (MaCTBH, MaBH,
NoiDung, TinhTrang) VALUES (?, ?, ?, ?)";

        PreparedStatement pstCT = conn.prepareStatement(sqlCT);
        PreparedStatement pstUpImei =
conn.prepareStatement(sqlUpdateImei);
        PreparedStatement pstInsBH = conn.prepareStatement(sqlInsertBH);
        PreparedStatement pstGetImei =
conn.prepareStatement(sqlSelectImei);
        PreparedStatement pstInsCTBH =
conn.prepareStatement(sqlInsertCTBH);

```

```

        int currentNumBH =
Integer.parseInt(generateMaBaoHanh().substring(2));
        int currentNumCTBH = Integer.parseInt(new
ChiTietBaoHanhDAO().generateMaCTBH().substring(4));

        for (ChiTietPhieuXuat ct : dsChiTiet) {
            pstCT.setString(1, ct.getMaPhieuXuat());
            pstCT.setString(2, ct.getMaPhienBan());
            pstCT.setInt(3, ct.getSoLuong());
            pstCT.setDouble(4, ct.getDonGia());
            pstCT.addBatch();

            pstGetImei.setString(1, ct.getMaPhienBan());
            pstGetImei.setInt(2, ct.getSoLuong());
            ResultSet rsImei = pstGetImei.executeQuery();

            int countFound = 0;
            while (rsImei.next()) {
                countFound++;
                String imeiReal = rsImei.getString("MaImei");

                pstUpImei.setString(1, px.getMaPhieuXuat());
                pstUpImei.setString(2, imeiReal);
                pstUpImei.addBatch();

                String maBH = String.format("BH%03d", currentNumBH++);
                pstInsBH.setString(1, maBH);
                pstInsBH.setString(2, imeiReal);
                pstInsBH.setString(3, px.getMaPhieuXuat());
                pstInsBH.setTimestamp(4, px.getNgayXuat());

                java.util.Calendar cal = java.util.Calendar.getInstance();
                cal.setTime(px.getNgayXuat());
                cal.add(java.util.Calendar.MONTH, 12);
                pstInsBH.setDate(5, new
java.sql.Date(cal.getTimeInMillis()));
                pstInsBH.addBatch();

                String maCTBH = "CT" + maBH;
                pstInsCTBH.setString(1, maCTBH);
                pstInsCTBH.setString(2, maBH);
                pstInsCTBH.setString(3, "Thiết bị mới xuất kho - Kích hoạt
bảo hành điện tử");
                pstInsCTBH.setString(4, "Còn bảo hành");
                pstInsCTBH.addBatch();
            }

            if (countFound < ct.getSoLuong()) {

```

```

        throw new Exception("Sản phẩm mã " + ct.getMaPhienBan() +
" không đủ máy trong kho!");
    }
}

pstCT.executeBatch();
pstUpImei.executeBatch();
pstInsBH.executeBatch();
pstInsCTBH.executeBatch();

conn.commit();
return true;
} catch (Exception e) {
    if (conn != null)
        try {
            conn.rollback();
        } catch (SQLException ex) {
        }
    throw new RuntimeException(e.getMessage());
}
}

public String generateMaBaoHanh() {
    String sql = "SELECT MaBH FROM BaoHanh ORDER BY CAST(SUBSTRING(MaBH,
3) AS UNSIGNED) DESC LIMIT 1";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            String lastMa = rs.getString("MaBH");
            int num = Integer.parseInt(lastMa.substring(2));
            return String.format("BH%03d", num + 1);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "BH000";
}

public ArrayList<PhieuXuat> filterSQL(Timestamp from, Timestamp to, Double
min, Double max) {
    ArrayList<PhieuXuat> list = new ArrayList<>();
    String sql = "SELECT * FROM PhieuXuat WHERE NgayXuat BETWEEN ? AND ?
AND TongTien BETWEEN ? AND ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement pst = conn.prepareStatement(sql)) {
        pst.setTimestamp(1, from != null ? from : Timestamp.valueOf("1970-
01-01 00:00:00"));

```

```

        pst.setTimestamp(2, to != null ? to : new
Timestamp(System.currentTimeMillis()));
        pst.setDouble(3, min);
        pst.setDouble(4, max);
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

public double getDiscountPercentage(String maPX) {
    String sql = "SELECT km.PhanTramGiam FROM PhieuXuat px " +
        "JOIN KhuyenMai km ON px.MaKM = km.MaKM " +
        "WHERE px.MaPhieuXuat = ?";

    try (Connection conn = Backend.DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, maPX);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            return rs.getDouble("PhanTramGiam");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public String getTenKhachHang(String maKH) {
    String sql = "SELECT HoTen FROM ConNguoi WHERE ID = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, maKH);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            return rs.getString("HoTen");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "Khách vắng lai";
}
}

```

20. SanPhamDAO

```
public class SanPhamDAO implements DAOInterface<SanPham> {
    @Override
    public ArrayList<SanPham> selectAll() {
        ArrayList<SanPham> list = new ArrayList<>();
        String sql = "SELECT sp.*, " +
            "(SELECT COALESCE(SUM(pb.SoLuongTon), 0) " + " FROM PhienBanSP pb "
            " +
            " WHERE pb.MaSP = sp.MaSP AND pb.TrangThai = 1) AS TongSoLuong "
            +
            "FROM SanPham sp " + "WHERE sp.TrangThai = TRUE";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                list.add(new SanPham(
                    rs.getString("MaSP"),
                    rs.getString("TenSP"),
                    rs.getInt("TongSoLuong"),
                    rs.getString("MaLoai"),
                    rs.getString("MaHang"),
                    rs.getString("MoTa"),
                    rs.getInt("ThoiGianBaoHanh"),
                    rs.getBoolean("TrangThai")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

    @Override
    public SanPham selectById(String maSP) {
        String sql = "SELECT * FROM SanPham WHERE MaSP=? AND TrangThai=TRUE";
        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, maSP);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                return new SanPham(
                    rs.getString("MaSP"),
                    rs.getString("TenSP"),
                    rs.getInt("TongSoLuong"),
                    rs.getString("MaLoai"),
                    rs.getString("MaHang"),
                    rs.getString("MoTa"),
                    rs.getInt("ThoiGianBaoHanh"),
                    rs.getBoolean("TrangThai"));
            }
        }
    }
}
```

```

        rs.getBoolean("TrangThai"));
    }
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

@Override
public int insert(SanPham sp) {
    String sql="INSERT INTO SanPham
(MaSP,TenSP,MaLoai,MaHang,MoTa,ThoiGianBaoHanh) Value(?,?,?,?,?,?)";
    try(Connection conn=DatabaseHelper.getConnection();
        PreparedStatement stmt=conn.prepareStatement(sql)){
        stmt.setString(1,sp.getMaSP());
        stmt.setString(2,sp.getTenSP());
        stmt.setString(3,sp.getMaLoai());
        stmt.setString(4,sp.getMaHang());
        stmt.setString(5,sp.getMoTa());
        stmt.setInt(6,sp.getThoiGianBaoHanh());

        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

@Override
public int update(SanPham sp) {
    String sql="UPDATE SanPham SET TenSP=?, MaLoai=?, MaHang=?, MoTa=?,
ThoiGianBaoHanh=? WHERE MaSP=";
    try(Connection conn=DatabaseHelper.getConnection();
        PreparedStatement stmt=conn.prepareStatement(sql)){
        stmt.setString(1,sp.getTenSP());
        stmt.setString(2,sp.getMaLoai());
        stmt.setString(3,sp.getMaHang());
        stmt.setString(4,sp.getMoTa());
        stmt.setInt(5,sp.getThoiGianBaoHanh());
        stmt.setString(6,sp.getMaSP());

        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

```

```

@Override
public int delete(String id) {
    String sql="UPDATE SANPHAM SET TrangThai=FALSE WHERE MaSP=? ";
    try(Connection conn=DatabaseHelper.getConnection();
        PreparedStatement stmt=conn.prepareStatement(sql)){
        stmt.setString(1,id);
        return stmt.executeUpdate();
    }catch(Exception e){
        e.printStackTrace();
    }
    return 0;
}

// Lấy danh sách loại dưới dạng chuỗi "Mã - Tên"
public ArrayList<String> getDanhSachLoai() {
    ArrayList<String> list = new ArrayList<>();
    String sql = "SELECT MaLoai, TenLoai FROM LoaiSP ";

    // Sử dụng try-with-resources đồng bộ với các hàm trên
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            String ma = rs.getString("MaLoai");
            String ten = rs.getString("TenLoai");
            list.add(ma + " - " + ten);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

// Lấy danh sách hãng dưới dạng chuỗi "Mã - Tên"
public ArrayList<String> getDanhSachHang() {
    ArrayList<String> list = new ArrayList<>();
    String sql = "SELECT MaHang, TenHang FROM HangSX ";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            String ma = rs.getString("MaHang");
            String ten = rs.getString("TenHang");
            list.add(ma + " - " + ten);
        }
    }
}

```



```

    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

public String getNextID() {
    String sql = "SELECT MaSP FROM SanPham";
    int maxId = 0;

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            String maSP = rs.getString("MaSP");
            if (maSP != null) {
                try {
                    int id = Integer.parseInt(maSP.replaceAll("[^0-9]",
""));

                    if (id > maxId) {
                        maxId = id;
                    }
                } catch (Exception e) {
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "SP" + String.format("%03d", maxId + 1);
}
}

```

21. PhienBanSanPhamDAO

```

public class PhienBanSanPhamDAO implements DAOInterface<PhienBanSanPham> {

    @Override
    public ArrayList<PhienBanSanPham> selectAll() {
        ArrayList<PhienBanSanPham> list = new ArrayList<>();
        String sql = "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " + "JOIN
SanPham sp ON pb.MaSP = sp.MaSP "
            + "WHERE pb.TrangThai = TRUE";
    }
}

```

```

try (Connection conn = DatabaseHelper.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        PhienBanSanPham pbsp = new PhienBanSanPham();
        pbsp.setMaPhienBan(rs.getString("MaPhienBan"));
        pbsp.setMaSP(rs.getString("MaSP"));
        pbsp.setMauSac(rs.getString("MauSac"));
        pbsp.setCongSuat(rs.getString("CongSuat"));
        pbsp.setPin(rs.getString("Pin"));
        pbsp.setKetNoi(rs.getString("KetNoi"));
        pbsp.setGiaNhap(rs.getDouble("GiaNhap"));
        pbsp.setGiaBan(rs.getDouble("GiaBan"));
        pbsp.setSoLuongTon(rs.getInt("SoLuongTon"));
        pbsp.setTrangThai(rs.getBoolean("TrangThai"));
        pbsp.setHinhAnh(rs.getString("HinhAnh"));

        pbsp.setTenSP(rs.getString("TenSP"));
        list.add(pbsp);
    }
} catch (Exception e) {
    e.printStackTrace();
}
return list;
}

@Override
public PhienBanSanPham selectById(String maPhienBan) {
    String sql = "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "WHERE pb.MaPhienBan = ? AND pb.TrangThai = TRUE";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maPhienBan);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            PhienBanSanPham pb = new PhienBanSanPham(
                rs.getString("MaPhienBan"),
                rs.getString("MaSP"),
                rs.getString("MauSac"),
                rs.getString("CongSuat"),
                rs.getString("Pin"),
                rs.getString("KetNoi"),
                rs.getDouble("GiaNhap"),
                rs.getDouble("GiaBan"),
                rs.getInt("SoLuongTon"),
                rs.getBoolean("TrangThai"),
            );
        }
    }
}

```

```

        rs.getString("HinhAnh"));

        pb.setTenSP(rs.getString("TenSP"));
        return pb;
    }
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

@Override
public int insert(PhienBanSanPham pbsp) {
    int result = 0;
    String sql = "INSERT INTO PhienBanSP VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, pbsp.getMaPhienBan());
        stmt.setString(2, pbsp.getMaSP());
        stmt.setString(3, pbsp.getMauSac());
        stmt.setString(4, pbsp.getCongSuat());
        stmt.setString(5, pbsp.getPin());
        stmt.setString(6, pbsp.getKetNoi());
        stmt.setDouble(7, pbsp.getGiaNhap());
        stmt.setDouble(8, pbsp.getGiaBan());
        stmt.setInt(9, pbsp.getSoLuongTon());
        stmt.setBoolean(10, true);
        stmt.setString(11, pbsp.getHinhAnh());

        result = stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return result;
}

@Override
public int update(PhienBanSanPham pbsp) {
    String sql = "UPDATE PhienBanSP SET MauSac=?, CongSuat=?, Pin=?, KetNoi=?, GiaNhap=?, GiaBan=?, SoLuongTon=?, HinhAnh=? WHERE MaPhienBan=?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, pbsp.getMauSac());
        stmt.setString(2, pbsp.getCongSuat());
        stmt.setString(3, pbsp.getPin());
        stmt.setString(4, pbsp.getKetNoi());
        stmt.setDouble(5, pbsp.getGiaNhap());

```

```

        stmt.setDouble(6, pbsp.getGiaBan());
        stmt.setInt(7, pbsp.getSoLuongTon());
        stmt.setString(8, pbsp.getHinhAnh());
        stmt.setString(9, pbsp.getMaPhienBan());

        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

@Override
public int delete(String id) {
    String sql = "UPDATE PhienBanSP SET TrangThai=FALSE WHERE MaPhienBan=?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, id);
        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public int deleteByMaSP(String maSP) {
    String sql = "UPDATE PhienBanSP SET TrangThai = FALSE WHERE MaSP = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maSP);
        return stmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

// Chọn theo nhà cung cấp
public ArrayList<PhienBanSanPham> selectByNCC(String maNCC) {
    ArrayList<PhienBanSanPham> list = new ArrayList<>();
    String sql = "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "JOIN NCC_SanPham ncc_sp ON sp.MaSP = ncc_sp.MaSP " +
        "WHERE ncc_sp.MaNCC = ?";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

```

```

        stmt.setString(1, maNCC);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            PhienBanSanPham pb = new PhienBanSanPham(
                rs.getString("MaPhienBan"),
                rs.getString("MaSP"),
                rs.getString("MauSac"),
                rs.getString("CongSuat"),
                rs.getString("Pin"),
                rs.getString("KetNoi"),
                rs.getDouble("GiaNhap"),
                rs.getDouble("GiaBan"),
                rs.getInt("SoLuongTon"),
                rs.getBoolean("TrangThai"),
                rs.getString("HinhAnh"));
            pb.setTenSP(rs.getString("TenSP"));
            list.add(pb);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

// Lấy Phiên Bản từ mã sản phẩm
public ArrayList<PhienBanSanPham> selectByMaSP(String maSP) {
    ArrayList<PhienBanSanPham> list = new ArrayList<>();
    String sql = "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "WHERE pb.MaSP = ? AND pb.TrangThai = TRUE";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, maSP);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            PhienBanSanPham pb = new PhienBanSanPham(
                rs.getString("MaPhienBan"),
                rs.getString("MaSP"),
                rs.getString("MauSac"),
                rs.getString("CongSuat"),
                rs.getString("Pin"),
                rs.getString("KetNoi"),
                rs.getDouble("GiaNhap"),
                rs.getDouble("GiaBan"),
                rs.getInt("SoLuongTon"),

```

```

        rs.getBoolean("TrangThai"),
        rs.getString("HinhAnh"));
        pb.setTenSP(rs.getString("TenSP"));
        list.add(pb);
    }
} catch (Exception e) {
    e.printStackTrace();
}
return list;
}

public String getNextID() {
    String sql = "SELECT MaPhienBan FROM PhienBanSP";
    int maxId = 0;

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            String maPB = rs.getString("MaPhienBan");
            if (maPB != null) {
                try {
                    int id = Integer.parseInt(maPB.replaceAll("[^0-9]",
""));

                    if (id > maxId) {
                        maxId = id;
                    }
                } catch (Exception e) {
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "PB" + String.format("%03d", maxId + 1);
}

public ArrayList<PhienBanSanPham> selectByLoai(String maLoai) {
    ArrayList<PhienBanSanPham> list = new ArrayList<>();
    String sql = "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "WHERE sp.MaLoai = ? AND pb.TrangThai = TRUE";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, maLoai);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {

```

```

        PhienBanSanPham pb = new PhienBanSanPham();
        pb.setMaPhienBan(rs.getString("MaPhienBan"));
        pb.setMaSP(rs.getString("MaSP"));
        pb.setTenSP(rs.getString("TenSP"));
        pb.setMauSac(rs.getString("MauSac"));

        pb.setGiaBan(rs.getDouble("GiaBan"));
        pb.setHinhAnh(rs.getString("HinhAnh"));

        pb.setGiaNhap(rs.getDouble("GiaNhap"));
        pb.setSoLuongTon(rs.getInt("SoLuongTon"));
        pb.setTrangThai(rs.getBoolean("TrangThai"));
        list.add(pb);
    }
} catch (Exception e) {
    e.printStackTrace();
}
return list;
}

public ArrayList<PhienBanSanPham> selectByFilterNhapHang(String maNCC,
String maLoai, String query) {
    ArrayList<PhienBanSanPham> list = new ArrayList<>();
    StringBuilder sql = new StringBuilder(
        "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP "
    );

    if (!maNCC.equals("All"))
        sql.append(" JOIN NCC_SanPham ncc_sp ON sp.MaSP = ncc_sp.MaSP");
    sql.append(" WHERE pb.TrangThai = TRUE");
    if (!maNCC.equals("All"))
        sql.append(" AND ncc_sp.MaNCC = ?");
    if (!maLoai.equals("All"))
        sql.append(" AND sp.MaLoai = ?");
    if (query != null && !query.trim().isEmpty()) {
        String[] words = query.toLowerCase().split("\\s+");
        for (int i = 0; i < words.length; i++) {
            sql.append(" AND (LOWER(sp.TenSP) LIKE ? OR
LOWER(pb.MaPhienBan) LIKE ? " +
                "OR LOWER(pb.MauSac) LIKE ? OR CAST(pb.GiaNhap AS
CHAR) LIKE ? " +
                "OR CAST(pb.SoLuongTon AS CHAR) LIKE ?)");
        }
    }

    try (Connection conn = DatabaseHelper.getConnection());

```

```

        PreparedStatement stmt =
conn.prepareStatement(sql.toString()) {
    int index = 1;
    if (!maNCC.equals("All"))
        stmt.setString(index++, maNCC);
    if (!maLoai.equals("All"))
        stmt.setString(index++, maLoai);
    if (query != null && !query.trim().isEmpty()) {
        String[] words = query.toLowerCase().split("\\s+");
        for (String word : words) {
            String p = "%" + word + "%";
            stmt.setString(index++, p);
            stmt.setString(index++, p);
            stmt.setString(index++, p);
            stmt.setString(index++, p);
            stmt.setString(index++, p);
        }
    }
}

ResultSet rs = stmt.executeQuery();
while (rs.next()) {
    PhienBanSanPham pb = new PhienBanSanPham(
        rs.getString("MaPhienBan"), rs.getString("MaSP"),
rs.getString("MauSac"),
        rs.getString("CongSuat"), rs.getString("Pin"),
rs.getString("KetNoi"),
        rs.getDouble("GiaNhap"), rs.getDouble("GiaBan"),
rs.getInt("SoLuongTon"),
        rs.getBoolean("TrangThai"), rs.getString("HinhAnh"));
    pb.setTenSP(rs.getString("TenSP"));
    list.add(pb);
}
} catch (Exception e) {
    e.printStackTrace();
}
return list;
}

public ArrayList<PhienBanSanPham> selectByFilterBanHang(String maLoai,
String query) {
    ArrayList<PhienBanSanPham> list = new ArrayList<>();
    StringBuilder sql = new StringBuilder(
        "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP "
    );

    sql.append(" WHERE pb.TrangThai = TRUE");
    if (!maLoai.equals("All"))

```



```

        sql.append(" AND sp.MaLoai = ?");
        if (query != null && !query.trim().isEmpty()) {
            String[] words = query.toLowerCase().split("\\s+");
            for (int i = 0; i < words.length; i++) {
                sql.append(" AND (LOWER(sp.TenSP) LIKE ? OR
LOWER(pb.MaPhienBan) LIKE ? " +
                    "OR LOWER(pb.MauSac) LIKE ? OR CAST(pb.GiaBan AS CHAR)
LIKE ? " +
                    "OR CAST(pb.SoLuongTon AS CHAR) LIKE ?)");
            }
        }

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql.toString())) {
            int index = 1;
            if (!maLoai.equals("All"))
                stmt.setString(index++, maLoai);
            if (query != null && !query.trim().isEmpty()) {
                String[] words = query.toLowerCase().split("\\s+");
                for (String word : words) {
                    String p = "%" + word + "%";
                    stmt.setString(index++, p);
                    stmt.setString(index++, p);
                    stmt.setString(index++, p);
                    stmt.setString(index++, p);
                    stmt.setString(index++, p);
                }
            }

            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                PhienBanSanPham pb = new PhienBanSanPham(
                    rs.getString("MaPhienBan"), rs.getString("MaSP"),
                    rs.getString("MauSac"),
                    rs.getString("CongSuat"), rs.getString("Pin"),
                    rs.getString("KetNoi"),
                    rs.getDouble("GiaNhap"), rs.getDouble("GiaBan"),
                    rs.getInt("SoLuongTon"),
                    rs.getBoolean("TrangThai"), rs.getString("HinhAnh"));
                pb.setTenSP(rs.getString("TenSP"));
                list.add(pb);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return list;
    }

```

```

public ArrayList<PhienBanSanPham> search(String text) {
    ArrayList<PhienBanSanPham> list = new ArrayList<>();
    String sql = "SELECT pb.*, sp.TenSP FROM PhienBanSP pb " +
        "JOIN SanPham sp ON pb.MaSP = sp.MaSP " +
        "WHERE pb.TrangThai = TRUE AND (pb.MaPhienBan LIKE ? OR
sp.TenSP LIKE ? OR pb.MauSac LIKE ?)";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        String query = "%" + text + "%";
        stmt.setString(1, query);
        stmt.setString(2, query);
        stmt.setString(3, query);

        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            PhienBanSanPham pbsp = new PhienBanSanPham();
            pbsp.setMaPhienBan(rs.getString("MaPhienBan"));
            pbsp.setMaSP(rs.getString("MaSP"));
            pbsp.setMauSac(rs.getString("MauSac"));
            pbsp.setCongSuat(rs.getString("CongSuat"));
            pbsp.setPin(rs.getString("Pin"));
            pbsp.setKetNoi(rs.getString("KetNoi"));
            pbsp.setGiaNhap(rs.getDouble("GiaNhap"));
            pbsp.setGiaBan(rs.getDouble("GiaBan"));
            pbsp.setSoLuongTon(rs.getInt("SoLuongTon"));
            pbsp.setTrangThai(rs.getBoolean("TrangThai"));
            pbsp.setHinhAnh(rs.getString("HinhAnh"));
            pbsp.setTenSP(rs.getString("TenSP"));

            list.add(pbsp);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}
}

```

22. TaiKhoanDAO

```

public class TaiKhoanDAO {

    public TaiKhoan login(String username, String password) {
        TaiKhoan tk = null;
    }
}

```

```

String sql = ""
        SELECT TenDangNhap, MatKhai, MaNV, MaNhomQuyen, TrangThai
        FROM TaiKhoan
        WHERE TenDangNhap = ? AND MatKhai = ? AND TrangThai = 1
        """;

try (
    Connection con = DatabaseHelper.getConnection();
    PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, username);
    ps.setString(2, password);
    ResultSet rs = ps.executeQuery();
    if (rs.next()) {
        tk = new TaiKhoan();
        tk.setUsername(rs.getString("TenDangNhap"));
        tk.setPassword(rs.getString("MatKhai"));
        tk.setMaNV(rs.getString("MaNV"));
        tk.setMaNhomQuyen(rs.getString("MaNhomQuyen"));
        tk.setStatus(rs.getBoolean("TrangThai") ? 1 : 0);
    }
} catch (Exception e) {
    e.printStackTrace();
}
return tk;
}

public TaiKhoan getTaiKhoanByMaNV(String maNV) {
    TaiKhoan tk = null;
    String sql = "SELECT TenDangNhap, MatKhai, MaNV, MaNhomQuyen,
TrangThai FROM TaiKhoan WHERE MaNV = ?";

    try (
        Connection con = DatabaseHelper.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, maNV);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            tk = new TaiKhoan();
            tk.setUsername(rs.getString("TenDangNhap"));
            tk.setPassword(rs.getString("MatKhai"));
            tk.setMaNV(rs.getString("MaNV"));
            tk.setMaNhomQuyen(rs.getString("MaNhomQuyen"));
            tk.setStatus(rs.getBoolean("TrangThai") ? 1 : 0);
        }
    } catch (Exception e) {
        System.err.println("Lỗi lấy tài khoản theo mã NV: " +
e.getMessage());
        e.printStackTrace();
    }
}

```

```

    }
    return tk;
}

public int insert(TaiKhoan tk) {
    String sql = "INSERT INTO TaiKhoan (TenDangNhap, MatKhai, MaNV, MaNhomQuyen, TrangThai) VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, tk.getUsername());
        ps.setString(2, tk.getPassword());
        ps.setString(3, tk.getMaNV());
        ps.setString(4, tk.getMaNhomQuyen());
        ps.setInt(5, tk.getStatus());
        return ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        return 0;
    }
}

public boolean existsByUsername(String username) {
    String sql = "SELECT COUNT(*) FROM TaiKhoan WHERE TenDangNhap = ?";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, username);
        ResultSet rs = ps.executeQuery();
        if (rs.next())
            return rs.getInt(1) > 0;
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
}

```

23. ThongKeDAO

```

public class ThongKeDAO {
    public double getDoanhThuNgay() {
        String sql = "SELECT SUM(TongTien) FROM PhieuXuat WHERE DATE(NgayXuat) = CURDATE() AND TrangThai = TRUE";

        try (Connection conn = DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

```

```

        if(rs.next()) return rs.getDouble(1);
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}

public double getVonNhapThang() {
    String sql = "SELECT SUM(TongTien) FROM PhieuNhap " +
        "WHERE MONTH(NgayNhap) = MONTH(CURDATE()) " +
        "AND YEAR(NgayNhap) = YEAR(CURDATE()) AND TrangThai =
TRUE";
    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) return rs.getDouble(1);
    } catch (Exception e) { e.printStackTrace(); }
    return 0;
}

public int getDonHangMoiNgay() {
    String sql = "SELECT COUNT(*) FROM PhieuXuat WHERE DATE(NgayXuat) =
CURDATE() AND TrangThai = TRUE";
    try (Connection conn = Backend.DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) return rs.getInt(1);
    } catch (Exception e) { e.printStackTrace(); }
    return 0;
}

public double getLoiNhuanThang() {
    // Lợi nhuận = Tổng tiền xuất - (Tổng tiền nhập của các SP đã bán)
    // Để đơn giản nhất, ta tính hiệu số thu chi trong tháng
    String sql = "SELECT (SELECT SUM(TongTien) FROM PhieuXuat WHERE
MONTH(NgayXuat) = MONTH(CURDATE()) AND TrangThai = TRUE) - " +
        "(SELECT SUM(TongTien) FROM PhieuNhap WHERE
MONTH(NgayNhap) = MONTH(CURDATE()) AND TrangThai = TRUE)";
    try (Connection conn = Backend.DatabaseHelper.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) return rs.getDouble(1);
    } catch (Exception e) { e.printStackTrace(); }
    return 0;
}

public void getTop5SanPham(DefaultCategoryDataset dataset, String type) {
    dataset.clear();

```

```

        String timeCondition = "";
        if (type.equals("Ngày")) timeCondition = "DATE(px.NgayXuat) =
CURDATE()";
        else if (type.equals("Tháng")) timeCondition = "MONTH(px.NgayXuat) =
MONTH(CURDATE()) AND YEAR(px.NgayXuat) = YEAR(CURDATE())";
        else if (type.equals("Năm")) timeCondition = "YEAR(px.NgayXuat) =
YEAR(CURDATE())";

        String sql = "SELECT s.TenSP, SUM(ctx.SoLuong) as TongSL " +
            "FROM ChiTietPhieuXuat ctx " +
            "JOIN PhienBanSP pb ON ctx.MaPhienBan = pb.MaPhienBan " +
            "JOIN SanPham s ON pb.MaSP = s.MaSP " +
            "JOIN PhieuXuat px ON ctx.MaPhieuXuat = px.MaPhieuXuat " +
            "WHERE " + timeCondition + " AND px.TrangThai = TRUE " +
            "GROUP BY s.TenSP ORDER BY TongSL DESC LIMIT 5";

        try (Connection conn = Backend.DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                dataset.addValue(rs.getInt("TongSL"), "Sản phẩm",
rs.getString("TenSP"));
            }
        } catch (Exception e) { e.printStackTrace(); }
    }

    public void getTiTrongDoanhThuLoai(DefaultPieDataset dataset, String type)
    {
        dataset.clear();

        String timeCondition = "";
        if (type.equals("Ngày")) timeCondition = "DATE(px.NgayXuat) =
CURDATE()";
        else if (type.equals("Tháng")) timeCondition = "MONTH(px.NgayXuat) =
MONTH(CURDATE()) AND YEAR(px.NgayXuat) = YEAR(CURDATE())";
        else if (type.equals("Năm")) timeCondition = "YEAR(px.NgayXuat) =
YEAR(CURDATE())";

        String sql = "SELECT l.TenLoai, SUM(ctx.ThanhTien) as DoanhThu " +
            "FROM ChiTietPhieuXuat ctx " +
            "JOIN PhienBanSP pb ON ctx.MaPhienBan = pb.MaPhienBan " +
            "JOIN SanPham s ON pb.MaSP = s.MaSP " +
            "JOIN LoaiSP l ON s.MaLoai = l.MaLoai " +
            "JOIN PhieuXuat px ON ctx.MaPhieuXuat = px.MaPhieuXuat " +
            "WHERE " + timeCondition + " AND px.TrangThai = TRUE " +
            "GROUP BY l.TenLoai";

        try (Connection conn = Backend.DatabaseHelper.getConnection();

```

```

        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery() {
            while (rs.next()) {
                dataset.setValue(rs.getString("TenLoai"),
rs.getDouble("DoanhThu"));
            }
        } catch (Exception e) { e.printStackTrace(); }
    }

    public void getXuHuongTrongThang(DefaultCategoryDataset dataset) {
        dataset.clear();
        // Lấy dữ liệu 15 ngày gần nhất
        String sql = "SELECT DATE_FORMAT(NgayXuat, '%d/%m') as Ngay, " +
            "COUNT(DISTINCT MaKH) as SLKhach, " +
            "COUNT(MaPhieuXuat) as SLDonHang " +
            "FROM PhieuXuat " +
            "WHERE NgayXuat >= DATE_SUB(CURDATE(), INTERVAL 15 DAY)
AND TrangThai = TRUE " +
            "GROUP BY Ngay ORDER BY NgayXuat ASC";

        try (Connection conn = Backend.DatabaseHelper.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                String ngay = rs.getString("Ngay");
                dataset.addValue(rs.getInt("SLKhach"), "Khách hàng", ngay);
                dataset.addValue(rs.getInt("SLDonHang"), "Đơn hàng", ngay);
            }
        } catch (Exception e) { e.printStackTrace(); }
    }
}

```

Tài liệu tham khảo

- [1] J. Gosling, B. Joy, G. Steele, G. Bracha, and A. Buckley, *The Java Language Specification, Java SE 8 Edition*. Boston, MA, USA: Addison-Wesley, 2014. [Online]. Available: <https://docs.oracle.com/javase/specs/jls/se8/jls8.pdf>
- [2] A. G. S. Sannidhi, V. S. S. Sastry, and S. G. S. Sastry, "A Survey on Java Desktop Application Development Frameworks," in *2021 International Conference on Computer Communication and Informatics (ICCCI)*, Coimbatore, India, 2021, pp. 1-6. doi: [10.1109/ICCCI50826.2021.9402517](https://doi.org/10.1109/ICCCI50826.2021.9402517)
- [3] H. Zhang and W. Zheng, "Research on the Application of Java Swing Technology in Graphical User Interface," in *2019 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS)*, Changsha, China, 2019, pp. 583-586. doi: [10.1109/ICITBS.2019.00147](https://doi.org/10.1109/ICITBS.2019.00147)
- [4] Oracle Corporation, "Java Platform, Standard Edition (Java SE) Documentation," *Oracle Help Center*, 2023. [Online]. Available: <https://docs.oracle.com/en/java/javase/>
- [5] P. Giarrusso, "A Java-based framework for graphical user interface development," in *IEEE Software*, vol. 18, no. 4, pp. 102-105, July-Aug. 2001. doi: [10.1109/MS.2001.936224](https://doi.org/10.1109/MS.2001.936224)
- [6] Slide bài giảng ngôn ngữ lập trình Java của cô Nguyễn Thị Hồng Anh.
- [7] Nguyễn Văn Ba (2006), *Phân tích và thiết kế hệ thống thông tin*, Nhà xuất bản Đại học Quốc gia Hà Nội.

